

PROJET DE FIN DE

Module

Machine learning

En

Master DS&BD

Sous le thème

Novo Molecular Generation pour la conception de
molécules à propriétés ciblées

Présenté par :

HIBA MHADA

WIJDANE BASSIRY

Encadré par :

Mr. Ben Lahmer El Habib

Mme Oumaima Guendoul

Remerciement

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous exprimons notre profonde gratitude à notre professeur **M. Ben Lahmer El Habib**, pour la qualité de son enseignement et pour nous avoir transmis les bases solides du machine learning qui ont rendu ce projet possible.

Nous adressons également nos sincères remerciements à gratitude à notre encadrante **Mme Oumaima Guendoul**, pour son accompagnement bienveillant tout au long de ce projet. Sa disponibilité, ses conseils précieux et sa patience nous ont été d'une grande aide. Nous la remercions sincèrement de nous avoir guidées avec autant de sérieux et de générosité.

Nous remercions l'ensemble du corps enseignant de la **Faculté des Sciences Ben M'Sik** pour la formation de qualité qu'ils nous ont dispensée tout au long de ce module.

Une pensée particulière va à nos **familles**, qui nous ont soutenues et encouragées sans relâche, et à nos **camarades de master** pour les échanges, le soutien mutuel et les bons moments partagés.

Enfin, nous remercions toute personne ayant contribué, de près ou de loin, à la réalisation de ce travail.

Hiba MHADA et Wijdane BASSIRY

Dédicaces

Nous dédions notre travail :

À nos parents, pour tout ce qu'ils ont sacrifié pour nous, pour leur amour inconditionnel, leurs prières et leur soutien qui ne m'ont jamais manqué. Aucun mot ne pourra exprimer ce que je ressens pour vous.

À nos frères et sœurs, compagnons de route et de toutes les luttes. Votre aide, vos recommandations et votre compagnie quotidienne ont été d'un grand soutien. Vous avez su nous motiver, nous divertir quand c'était nécessaire, et nous rappeler qu'on n'était jamais seuls sur cette voie.

À notre encadrante Mme Oumaima Guendoul, pour sa confiance en moi et pour m'avoir accompagnée avec tant de bienveillance.

À nos amies pour les moments de partage, d'entraide et de rires qui ont rendu ce parcours plus léger et plus beau.

À tous nos professeurs, qui ont su éveiller en nous la curiosité, l'envie d'apprendre et le goût de l'effort, nous adressons notre profonde gratitude, Grâce à votre patience, votre minutie et votre dévouement, vous avez laissé votre empreinte sur notre cheminement et aidé à établir les fondements de notre futur. Je vous remercie de nous avoir transmis bien plus que des savoirs.

A tout ceux dont l'amour, la patience et les encouragements silencieux ont accompagné chaque étape de ce parcours. Qu'ils soient proches par leur présence ou simplement par la pensée, ils demeurent les fondations invisibles de nos réussites.

Résumé

Ce projet s'inscrit dans le domaine de la génération moléculaire de novo assistée par intelligence artificielle.

L'objectif principal est de construire un pipeline complet permettant de générer de nouvelles molécules candidates à l'activité pharmaceutique, de les filtrer, d'évaluer leur qualité et de prédire leur toxicité à l'aide de modèles de machine learning.

Pour ce faire, nous avons utilisé le dataset ZINC 250k, composé de 249 455 molécules drug-like. Après une phase d'exploration et de prétraitement des données, nous avons généré 3 000 nouvelles molécules via l'API NVIDIA NIM MolMIM, en utilisant l'algorithme d'optimisation CMA-ES. Les molécules générées ont ensuite été filtrées selon des critères de qualité stricts : validité chimique, filtres PAINS et SA Score. Deux modèles de classification ont été entraînés pour la prédiction de toxicité, à savoir Random Forest et XGBoost, atteignant respectivement une accuracy de 97,74 % et 98,78 %. L'espace chimique des molécules générées a été visualisé grâce à la technique de réduction de dimensionnalité UMAP. Enfin, une application web interactive développée avec Streamlit a été déployée pour explorer les résultats de manière intuitive.

Les résultats obtenus montrent que notre pipeline produit des molécules avec un taux de validité de 100 %, une nouveauté de 100 % par rapport au dataset ZINC original, et un score QED moyen de 0,823, surpassant les modèles de référence de la littérature tels que VAE, GAN et Diffusion.

Mots-clés : Génération moléculaire de novo, ZINC 250k, NVIDIA NIM MolMIM, RDKit, Machine Learning, Random Forest, XGBoost, UMAP, Drug-likeness, Toxicité, Streamlit.

Abstract

This project focuses on AI-assisted de novo molecular generation.

The main objective is to build a complete pipeline for generating new pharmaceutical candidate molecules, filtering them, evaluating their quality, and predicting their toxicity using machine learning models.

We used the ZINC 250k dataset of 249,455 drug-like molecules. After data exploration and preprocessing, 3,000 new molecules were generated via the NVIDIA NIM MolMIM API using the CMA-ES optimization algorithm. The generated molecules were filtered using strict quality criteria: chemical validity, PAINS filters, and SA Score. Two classification models were trained for toxicity prediction Random Forest and XGBoost achieving accuracies of 97.74% and 98.78% respectively. The chemical space was visualized using UMAP dimensionality reduction, and an interactive Streamlit web application was deployed to explore the results.

Our pipeline produces molecules with 100% validity, 100% novelty against the original ZINC dataset, and a mean QED score of 0.823, outperforming reference models such as VAE, GAN, and Diffusion.

Keywords: De novo molecular generation, ZINC 250k, NVIDIA NIM MolMIM, RDKit, Machine Learning, Random Forest, XGBoost, UMAP, Drug-likeness, Toxicity, Streamlit.

Table des matières

Remerciement.....	2
Dédicaces.....	3
Résumé.....	4
Abstract.....	5
LISTE DES FIGURES	9
LISTE DES TABLEAUX.....	11
LISTE DES ABRÉVIATIONS.....	12
INTRODUCTION GÉNÉRALE	13
CHAPITRE 1 : Contexte et problématique	14
1. Introduction	14
2. Contexte Général	14
2.1 Découverte traditionnelle des médicaments.....	14
2.2 Apport de l'Intelligence Artificielle.....	14
2.3 Présentation de projet.....	15
3. La problématique	15
4. Objectif du projet.....	16
5. Cahier de charge	17
5.1 Introduction.....	17
5.2 Les besoins fonctionnels	17
5.3 Les besoins non fonctionnels de projet.....	17
5.4 Architecture fonctionnelle du système	18
5.5 Langages de programmation et bibliothèques utilisées	18
5.6 Éditeurs et environnements de développement	22
6. Conclusion	23
CHAPITRE 2 : ETAT D'ART	24
1. Introduction	24
2. revue de littérature	24
2. Synthèse des résultats.....	26
3. Comparaison et exploration des articles	28
4.1 Répartition des approches utilisées dans les travaux de recherche étudiés.....	28
4.2 Évolution des approches dans le temps	29
4.3 Répartition des algorithmes utilisées dans les travaux de recherche étudiés.....	29
4.4 Comparaison des performances des algorithmes	30
4.5 Comparaison des jeux de données et des algorithmes.....	30
5 Comparaison des prétraitements selon le jeu de données	31

4. Conclusion	31
Chapitre 3 : Méthodologie et Mise en Œuvre du Pipeline de Génération Moléculaire	32
1. Introduction	32
2. Méthodologie.....	32
2.1 Jeu de donnée	32
2.2 Exploration de jeu de donnée.....	33
2.3 Prétraitement de jeu de donnée	37
2.4 Génération moléculaire avec MolMIM NIM.....	39
2.5 Post-filtrage des molécules générées	41
2.6 Prédiction de toxicité par Machine Learning.....	43
2.7 Évaluation et comparaison avec la littérature	45
3. Conclusion	46
Chapitre 4 : Déploiement et Application Interactive	47
1. Introduction	47
2. Outils utilisés.....	47
3. Présentation générale de l'interface	47
4. Vue globale et comparaison avec la littérature.....	47
5. Exploration des candidats moléculaires	48
6. Visualisation de l'espace latent.....	48
7. Prédiction de toxicité en temps réel.....	49
8. Explicabilité par SHAP et analyse des atomes.....	49
9. Module counterfactual.....	50
10. Tableaux des données	50
11.Conclusion	51
Chapitre 5 : Conception du Poster Scientifique	52
1. Introduction	52
2. Objectif du poster	52
3.Structure du poster	52
4. Outils de conception.....	52
5 Description détaillée du contenu du poster	52
5.1 Introduction et System Workflow	52
5.2 Objectifs du projet.....	53
5.3 Data Analysis et Feature Engineering	53
5.4 Comparaison des modèles ML.....	54
5.5 Outils et environnement technique	54
5.6 Évaluation vs Littérature	55

5.7 Dashboard interactif et XAI.....	56
6 Conclusion	56
Conclusion Générale.....	57
BIBLIOGRAPHIE	58

LISTE DES FIGURES

Figure 1-Découverte traditionnelle des médicaments	14
Figure 2-Découverte assistée par IA	15
Figure 3- architecture fonctionnelle de système	18
Figure 4- logo du python	18
Figure 5-streamlit	19
Figure 6- RDKit	19
Figure 7-SCKIT LEARN	20
Figure 8-xgboost	20
Figure 9-UMAP-learn	20
Figure 10-pandas	21
Figure 11-numpy	21
Figure 12- logo Joblib	21
Figure 13- logo anaconda	22
Figure 14- logo Jupyter Notebooks	22
Figure 15-Répartition des approches utilisées dans les travaux	28
Figure 16-Évolution des approches dans le temps	29
Figure 17-Répartition des algorithmes utilisées dans les articles	29
Figure 18-Comparaison des performances des algorithmes	30
Figure 19-Comparaison des jeux de données et des algorithmes	30
Figure 20-Comparaison des prétraitements selon le jeu de données	31
Figure 21-Méthodes proposées	32
Figure 22-apercu général de dataset	34
Figure 23-Distribution des propriétés physicochimiques	34
Figure 24-Détection des valeurs aberrantes	35
Figure 25-analyse de corrélation	36
Figure 26-Visualisation des molécules en 2D	36
Figure 27-nettoyage de smiles	37
Figure 28-suppression des valeurs aberrantes	37
Figure 29-Application du filtre de Lipinski	38
Figure 30-Normalisation des propriétés	38
Figure 31-Encodage des molécules en fingerprints	39
Figure 32-Sélection des features et split train/test	39
Figure 33-Configuration de l'API NVIDIA NIM	40
Figure 34-Génération batch des molécules	41
Figure 35-filtres PAINS	41
Figure 36-filtres SA Score	42
Figure 37-Résultats de filtrage	42
Figure 38-visualisation 2D des meilleures molécules	43
Figure 39-Création des labels de toxicité	43
Figure 40-extraction des features	44
Figure 41-comparaison des modèles	44
Figure 42-matrice de confusion	45
Figure 43-Prédiction sur les molécules générées	45
Figure 44-présentation générale de l'interface	47
Figure 45-Vue globale et comparaison avec la littérature	47

Figure 46-barre latérale des filtres dynamiques	48
Figure 47-Les molécules affichées sous forme de structures 2D avec leur score QED.	48
Figure 48-visualisation de l'espace latent	49
Figure 49-résultat avec mention sur pour molécule non toxique	49
Figure 50-résultat avec mention toxique pour molécule toxique	49
Figure 51-la structure 2D de la molécule avec les atomes suspects mis en évidence	50
Figure 52-graphique SHAP avec les barres rouges/vertes et la structure avec atomes suspects	50
Figure 53-tableau des données	50
Figure 54- le bloc Introduction + System Workflow	53
Figure 55-le bloc Objectifs	53
Figure 56- bloc Data Analysis & Feature Engineering du poster avec les histogrammes, l'UMAP et les structures 2D	54
Figure 57-bloc de Comparaison des modèles ML du poster avec le tableau et la matrice de confusion	54
Figure 58-blocs Outils & Environnement Technique du poster	55
Figure 59-bloc Évaluation vs Littérature du poster avec le tableau comparatif et le graphique en barres	55
Figure 60-bloc Dashboard interactif & XAI du poster	56
Figure 61-bloc de conclusion et discussion	56

LISTE DES TABLEAUX

Tableau 1-Outils et rôles dans le projet	15
Tableau 2-les besoins fonctionnels de projet	17
Tableau 3-les besoins non fonctionnels de projet	17
Tableau 4-synthèse de l'état d'art.....	26
Tableau 5-Description du jeu de données.....	33

LISTE DES ABRÉVIATIONS

IA : Intelligence Artificielle
ML : Machine Learning
VAE : Variational Autoencoder (Auto-encodeur variationnel)
GAN : Generative Adversarial Network (Réseau antagoniste génératif)
RL : Reinforcement Learning (Apprentissage par renforcement)
LLM : Large Language Model
RF : Random Forest
SVM : Support Vector Machine
NN : Neural Network
KNN : K-Nearest Neighbors
NIM : NVIDIA Inference Microservices
CMA-ES : Covariance Matrix Adaptation Evolution Strategy
SMILES : Simplified Molecular Input Line Entry System
QED : Quantitative Estimate of Drug-likeness
SAS / SA Score : Synthetic Accessibility Score
PAINS : Pan-Assay Interference Compounds
MW : Molecular Weight (poids moléculaire)
LogP : Logarithm of the Partition coefficient
TPSA : Topological Polar Surface Area
HBD : Hydrogen Bond Donor
HBA : Hydrogen Bond Acceptor
AUC-ROC : Area Under the Curve – Receiver Operating Characteristic
FP / FN : Faux Positives / Faux Negatives
IQR : Interquartile range
SHAP : Shapley Additive explanations
XAI : explainable Artificial Intelligence
API : Application Programming Interface
CSV : Comma-Separated Values

INTRODUCTION GÉNÉRALE

La découverte de nouveaux médicaments est un processus long, coûteux et semé d'obstacles. Il faut en moyenne entre 10 et 15 ans et plusieurs milliards de dollars pour qu'une molécule passe du stade de la recherche fondamentale à celui d'un médicament approuvé, avec un taux d'échec qui dépasse les 90 %. Face à ces défis, la communauté scientifique cherche constamment des approches nouvelles et plus efficaces pour accélérer ce processus.

C'est dans ce contexte que l'intelligence artificielle et le machine learning ont commencé à jouer un rôle de plus en plus important dans la chimie médicinale computationnelle. Parmi les approches les plus prometteuses, la génération moléculaire de novo permet de concevoir de toutes nouvelles structures chimiques, sans se limiter aux molécules déjà connues, en explorant un espace chimique estimé à plus de 10^{60} molécules possibles.

Notre projet s'inscrit dans cette dynamique. Réalisé dans le cadre du module Machine Learning du Master Big Data & Data Science à la Faculté des Sciences Ben M'Sik, il propose un pipeline complet allant de l'exploration des données moléculaires jusqu'au déploiement d'une application web interactive, en passant par la génération, le filtrage et la prédiction de toxicité des molécules.

Pour générer de nouvelles molécules, nous avons exploité l'API NVIDIA NIM MolMIM, un modèle génératif de pointe basé sur l'optimisation CMA-ES. Nous avons travaillé sur le dataset ZINC 250k, qui contient 249 455 molécules drug-like, et nous avons utilisé la bibliothèque RDKit pour tout ce qui concerne la manipulation et l'analyse des structures chimiques. La prédiction de toxicité a été réalisée à l'aide de deux modèles de machine learning supervisés : Random Forest et XGBoost.

On va diviser ce dernier en quatre chapitres. **Le premier chapitre**, présente le contexte général du projet, la problématique, les objectifs et le cahier des charges. **Le deuxième chapitre** propose une analyse de l'état de l'art à travers l'étude de quinze articles scientifiques dans le domaine de la génération moléculaire, de la prédiction de propriétés et de la toxicité in silico. **Le troisième chapitre**, détaille la réalisation technique complète du pipeline : exploration des données, prétraitement, génération moléculaire, filtrage, évaluation, prédiction de toxicité. **Le quatrième chapitre** concerne le déploiement de l'application web, et enfin **le cinquième chapitre** concernant la conception de poster

CHAPITRE 1 : Contexte et problématique

1. Introduction

L'industrie pharmaceutique cherche constamment à réduire le temps et le coût nécessaires au développement de nouveaux médicaments. Cependant, ce processus reste long, coûteux et marqué par un taux d'échec élevé. L'intelligence artificielle apporte aujourd'hui de nouvelles solutions pour accélérer cette découverte. Parmi elles, la génération moléculaire *de novo* permet de concevoir automatiquement de nouvelles molécules à partir de données existantes. Dans ce cadre, notre projet vise à développer une plateforme capable de générer, filtrer et évaluer des molécules candidates grâce aux techniques modernes d'intelligence artificielle.

2. Contexte Général

2.1 Découverte traditionnelle des médicaments

La découverte d'un nouveau médicament suit généralement plusieurs étapes successives, depuis la recherche initiale, identification de la cible, criblage des molécules, tests précliniques, essais cliniques jusqu'à la commercialisation.

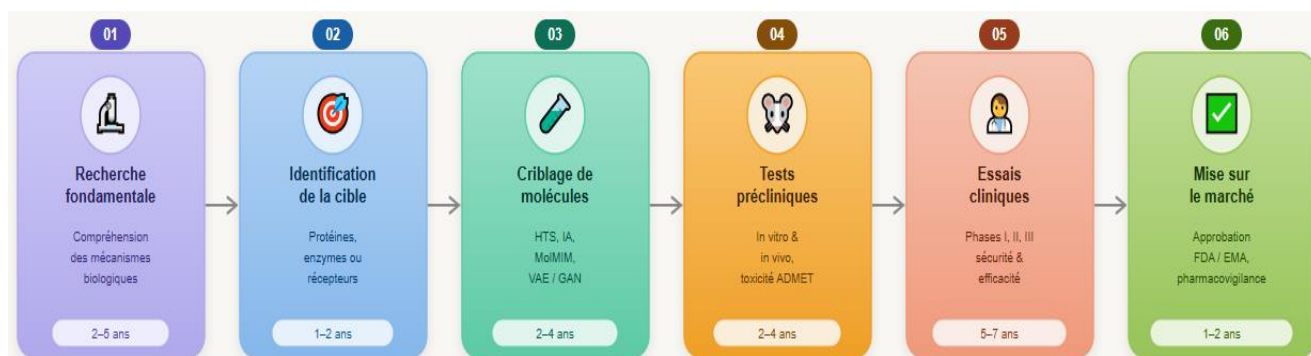


Figure 1-Découverte traditionnelle des médicaments

Cette approche présente toutefois des limites importantes. Le développement d'un médicament est très long, avec une durée moyenne de 10 à 15 ans, et nécessite des investissements très élevés pouvant atteindre 1 à 3 milliards de dollars. À cela s'ajoute un taux d'échec clinique supérieur à 90 %, ce qui rend le processus encore plus risqué. Face à ces contraintes, l'industrie pharmaceutique se tourne de plus en plus vers des solutions innovantes, notamment l'intelligence artificielle et les modèles génératifs, afin d'accélérer la découverte de nouvelles molécules tout en réduisant les coûts et les échecs.

2.2 Apport de l'Intelligence Artificielle

Grâce au Machine Learning, il est désormais possible d'explorer automatiquement un très grand nombre de molécules et de proposer des candidats prometteurs sans passer par toutes les étapes traditionnelles. Comme le schéma au-dessous montre l'IA apprend à créer de nouvelles molécules médicamenteuses, On lui donne d'abord une grande bibliothèque de molécules existantes, elle apprend leur "grammaire chimique", puis elle en invente de nouvelles toute seule. Enfin, on filtre automatiquement les meilleures candidates selon leurs propriétés biologiques.

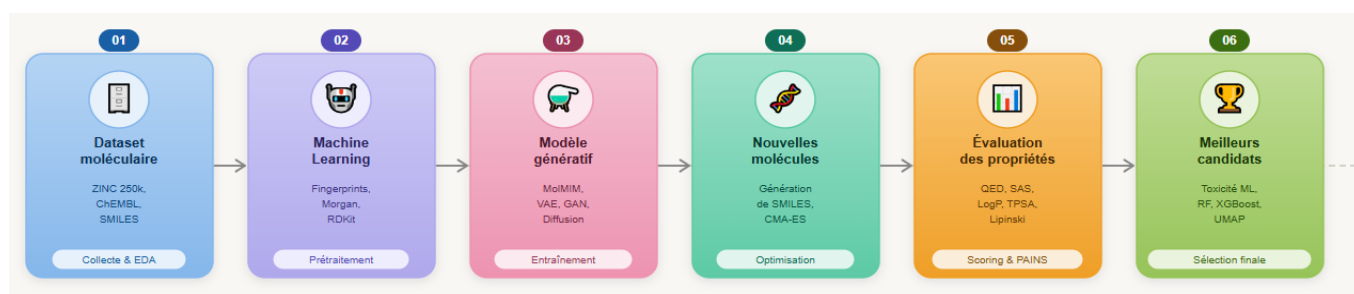


Figure 2-Découverte assistée par IA

2.3 Présentation de projet

Dans le cadre de ce projet, nous avons développé une chaîne complète de traitement dédiée à la génération et à l'analyse de molécules à l'aide de l'intelligence artificielle. L'objectif est d'intégrer plusieurs outils spécialisés afin de couvrir l'ensemble du pipeline : de la base de données moléculaires jusqu'à l'interface utilisateur.

Tableau 1-Outils et rôles dans le projet

Catégorie	Outil	Rôle
Dataset	ZINC 250k	Base de données contenant des molécules drug-like utilisées pour l'entraînement et l'analyse
Chémoinformatique	RDKit	Manipulation des structures moléculaires, calcul des descripteurs et validation chimique
Génération moléculaire	NVIDIA NIM MolMIM	Génération de nouvelles molécules via des modèles d'IA
Machine Learning	Scikit-Learn	Construction et entraînement des modèles prédictifs
Machine Learning avancé	XGBoost	Prédiction de la toxicité des molécules
Interface utilisateur	Streamlit	Développement d'une application web interactive pour visualiser et explorer les résultats

3. La problématique

Malgré les avancées récentes en intelligence artificielle appliquée à la chimie, plusieurs défis importants limitent encore la génération efficace de nouvelles molécules.

❖ Taille immense de l'espace chimique :

L'un des principaux problèmes est l'ampleur de l'espace chimique possible : Le nombre de molécules potentiellement existantes est gigantesque. On estime cet espace à environ **10⁶⁰ molécules possibles**. En comparaison, seules quelques millions de molécules sont connues ou répertoriées. Cela rend impossible une exploration exhaustive avec des méthodes classiques.

❖ Qualité des molécules générées

La génération de molécules ne suffit pas : elles doivent aussi être pertinentes et exploitables.

Une molécule valide doit être :

- Chimiquement stable et correcte
- Facile à synthétiser en laboratoire
- Peu toxique pour l'organisme
- Conforme aux règles de Lipinski (propriétés pharmacologiques)

❖ Problème de toxicité

Une grande partie des molécules candidates échoue lors des tests de sécurité.

- La toxicité est souvent difficile à détecter tôt
- Elle représente une cause majeure d'échec dans le développement de médicaments

Il est donc essentiel d'intégrer une prédiction de toxicité dès les premières étapes de génération.

❖ Interprétabilité des modèles d'IA

Les modèles génératifs utilisés sont souvent considérés comme des **boîtes noires**.

- Leur fonctionnement interne est difficile à comprendre
- Cela rend l'analyse des molécules générées complexe

Des techniques de visualisation comme **UMAP** sont nécessaires pour mieux interpréter et explorer les résultats.

Alors on pose la question suivante **Comment développer un pipeline intelligent capable de générer automatiquement de nouvelles molécules valides, innovantes et peu toxiques, à partir du dataset ZINC 250k ?**

4. Objectif du projet

Ce projet de fin d'études vise à atteindre les objectifs principaux suivants :

- Comprendre les enjeux liés à la découverte de nouveaux médicaments et les limites des approches traditionnelles dans l'exploration de l'espace chimique.
- Explorer et analyser le dataset ZINC 250k, en réalisant une étude statistique des propriétés physico-chimiques des molécules (logP, QED, TPSA, etc.).
- Mettre en place un pipeline de prétraitement des données moléculaires, incluant la canonicalisation des SMILES, le nettoyage des données, la suppression des doublons et le filtrage basé sur les règles de Lipinski.
- Générer de nouvelles molécules à l'aide de modèles d'intelligence artificielle, notamment via l'API NVIDIA NIM MolMIM et des méthodes d'optimisation avancées.
- Filtrer et améliorer la qualité des molécules générées en appliquant des critères chimiques tels que les filtres PAINS, le SA Score et la validité structurelle.
- Évaluer les performances du modèle génératif à travers des métriques standards telles que la validité, l'unicité, la nouveauté et le score QED.

- Développer des modèles de prédiction de toxicité à base de machine learning (Random Forest et XGBoost) afin d'estimer le risque des molécules générées.
- Visualiser l'espace chimique des molécules à l'aide de techniques de réduction de dimensionnalité comme UMAP pour mieux comprendre la distribution des structures générées.
- Concevoir une application web interactive avec Streamlit permettant l'exploration, le filtrage et la prédiction de toxicité des molécules en temps réel.

5. Cahier de charge

5.1 Introduction

Le présent cahier des charges décrit les besoins fonctionnels, non fonctionnels, les contraintes techniques ainsi que le périmètre du projet. Il définit les exigences du système de génération et d'analyse de molécules basé sur le dataset **ZINC 250k** et des modèles de machine learning et de génération moléculaire.

5.2 Les besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités attendues du système.

Tableau 2-les besoins fonctionnels de projet

Fonctionnalité	Description
Chargement des données	Importer et afficher le dataset ZINC 250k depuis CSV
Analyse exploratoire	Visualisation des propriétés (MW, LogP, TPSA, HBD, HBA)
Prétraitement	Nettoyage, canonicalisation SMILES, filtre Lipinski, fingerprints
Génération moléculaire	Générer des molécules via NVIDIA NIM MolMIM (CMA-ES)
Filtrage qualité	Filtrer via PAINS, SA Score < 4, validation chimique
Évaluation métriques	Calcul validité, unicité, nouveauté, QED
Visualisation UMAP	Réduction dimensionnelle et projection 2D de l'espace chimique
Prédiction toxicité	Modèles RF et XGBoost + prédiction via SMILES
Application web	Interface Streamlit interactive
Export données	Téléchargement CSV des molécules filtrées

5.3 Les besoins non fonctionnels de projet

Les besoins non fonctionnels définissent les contraintes de qualité.

Tableau 3-les besoins non fonctionnels de projet

Catégorie	Exigence
Performance	Traitement du dataset < 30 minutes
Scalabilité	Support jusqu'à 1 million de molécules
Reproductibilité	Random_state fixer (42)
Fiabilité	Validité molécules > 95%
Utilisabilité	Interface intuitive Streamlit
Maintenabilité	Code modulaire et commenté
Portabilité	Windows, Linux, MacOS, Colab

5.4 Architecture fonctionnelle du système

Le système est structuré en trois couches qui communiquent entre elles. La couche donnée constitue la base du système : elle stocke le dataset ZINC 250k, les molécules générées, les fingerprints encodés et les modèles ML entraînés. Au-dessus, la couche logique métier regroupe quatre modules qui assurent le traitement : le prétraitement des données, la génération de nouvelles molécules via l'API NVIDIA NIM, la prédiction de toxicité par Random Forest et XGBoost, et la visualisation UMAP. En haut de cette architecture, l'application Streamlit offre à l'utilisateur une interface web interactive pour explorer, filtrer et télécharger les résultats

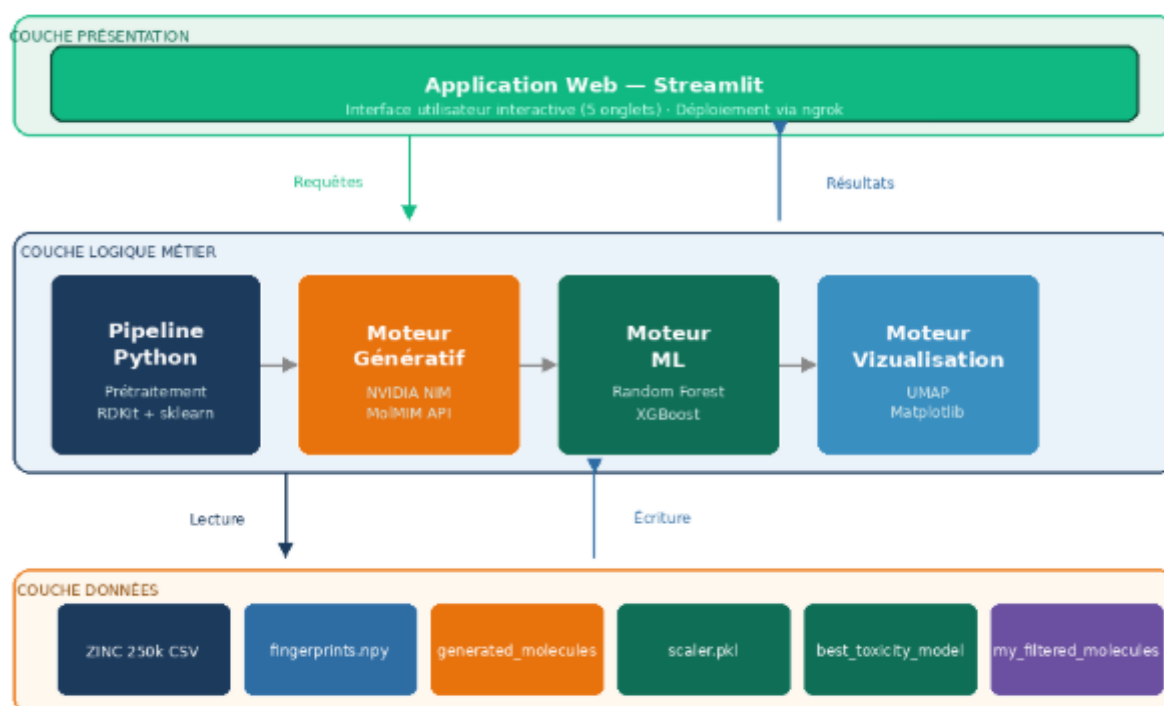


Figure 3- architecture fonctionnelle de système

5.5 Langues de programmation et bibliothèques utilisées

Dans le cadre de la conception de notre système, plusieurs langages et bibliothèques ont été utilisés pour répondre aux besoins fonctionnels et esthétiques de l'application. Voici une présentation détaillée de ces éléments :

Python :

- **Raison d'utilisation** : Python a été le langage central de notre projet en raison de sa simplicité, sa polyvalence, et la richesse de son écosystème de bibliothèques adaptées à la gestion des données, l'IA et le développement web.
- **Applications** :

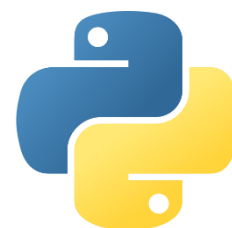


Figure 4- logo du python

- ✓ **Manipulation de données** : Grâce à des bibliothèques comme pandas, nous avons pu effectuer des traitements de données complexes de manière fluide et rapide.
- ✓ **Modélisation et apprentissage automatique** : Les modèles pré-entraînés ont été chargés à l'aide de job lib, une bibliothèque simple et performante pour la sérialisation des objets Python, permettant de sauvegarder et de charger facilement des modèles d'apprentissage automatique.

Streamlit :

- **Raison d'utilisation** : Streamlit est un framework Python populaire pour la création rapide d'applications web interactives. Il est parfaitement adapté pour des projets de data science où la visualisation des données et l'interaction avec l'utilisateur sont cruciales.
- **Applications** :
 - ✓ **Interface utilisateur** : Streamlit offre une interface très intuitive pour créer des composants comme des boutons, des champs de texte, des graphiques interactifs et des tableaux.
 - ✓ **Performance et rapidité** : L'intégration de ces composants se fait en quelques lignes de code, ce qui permet un prototypage rapide et un développement agile.



Figure 5-streamlit

RDKit :

- **Raison d'utilisation** : RDKit est la bibliothèque de chémoinformatique de référence en Python. Elle est indispensable pour manipuler les structures moléculaires, calculer des descripteurs et appliquer des filtres chimiques.
- **Applications** :
 - ✓ **Calcul des descripteurs** : Nous l'avons utilisé pandas pour canonicaliser les SMILES et calculer les propriétés physico-chimiques comme MW, LogP, TPSA, HBD et HBA sur les 249 455 molécules du dataset.
 - ✓ **Filtrage moléculaire** : Elle a permis d'appliquer les filtres PAINS, de calculer le SA Score de chaque molécule générée et de produire les visualisations 2D des structures moléculaires.



Figure 6- RDKit

NVIDIA NIM MolMIM :

- **Raison d'utilisation** : Cette API développée par NVIDIA est un modèle génératif de pointe qui optimise des molécules sources via l'algorithme CMA-ES pour produire de nouveaux candidats drug-like.
- **Applications** :
 - ✓ **Génération moléculaire** : Elle a été appelée en batch sur 1 000 molécules sources pour générer 3 000 nouvelles molécules candidates avec un score QED optimisé

- ✓ **Scoring automatique** : Elle a attribué automatiquement un score QED à chaque molécule générée, permettant de classer et sélectionner les meilleures candidates.

Scikit-learn:

- **Raison d'utilisation** : scikit-learn est la bibliothèque de machine learning de référence en Python, offrant des outils complets pour l'entraînement, l'évaluation et la normalisation des modèles.
- **Applications** :



Figure 7-SCKIT LEARN

- ✓ **Entraînement Random Forest** Elle a été utilisée pour entraîner le modèle Random Forest sur les 2 299 molécules du jeu d'entraînement, atteignant une accuracy de 97,74 % et un AUC-ROC de 0,99.
- ✓ **Normalisation et split** : Elle a permis de normaliser les propriétés physico-chimiques via StandardScaler et de réaliser le découpage stratifié du dataset en train/test.

XGBoost :

- **Raison d'utilisation** : XGBoost est un algorithme de gradient boosting reconnu pour sa rapidité et sa précision sur les données tabulaires, ce qui en fait un excellent candidat pour la classification de toxicité.
- **Applications** :

XGBoost

Figure 8-xgboost

- ✓ **Classification de toxicité** : Il a permis d'entraîner un classifieur atteignant 98,78 % d'accuracy, légèrement supérieur au Random Forest sur cette métrique.
- ✓ **Comparaison des modèles** : Il a été comparé au Random Forest selon les critères Accuracy, AUC-ROC et F1-Score pour sélectionner le meilleur modèle à déployer.

UMAP-learn:

- **Raison d'utilisation** : UMAP est une technique de réduction de dimensionnalité moderne capable de projeter des données de très haute dimension en 2D tout en préservant la structure locale des données.
- **Applications** :



Figure 9-UMAP-learn

- ✓ **Réduction dimensionnelle** : Il a réduit les Morgan Fingerprints de 2048 dimensions à 2 dimensions pour représenter visuellement l'espace chimique des 2 874 molécules filtrées.
- ✓ **Visualisation des clusters** : Il a produit un graphe de dispersion coloré par score QED, permettant d'identifier les clusters de molécules les plus prometteuses dans l'espace latent.

Pandas :

- **Raison d'utilisation** : Pandas est une bibliothèque incontournable pour la manipulation de données tabulaires, offrant une API simple et puissante pour travailler avec des DataFrames de grande taille.



Figure 10-pandas

- **Applications** :
- ✓ **Chargement et nettoyage** : Il a servi à charger, nettoyer et filtrer le dataset ZINC 250k de 249 455 molécules, en supprimant les SMILES invalides et les outliers.
- ✓ **Export des résultats** : Il a permis de gérer et d'exporter les différents fichiers CSV produits tout au long du pipeline, comme generated_molecules.csv et my_filtered_molecules.csv.

NumPy :

- **Raison d'utilisation** : NumPy est la bibliothèque de calcul numérique de référence en Python, indispensable pour manipuler des tableaux multidimensionnels de grande taille de manière efficace.



Figure 11-numpy

- **Applications** :
- ✓ **Stockage des fingerprints** : Il a servi à stocker et manipuler la matrice de fingerprints moléculaires de dimensions $235\,652 \times 2048$ bits sauvegardée dans fingerprints.npy.
- ✓ **Calculs vectorisés** : Il a permis de réaliser les calculs vectorisés lors de la construction des 1 034 features utilisées pour entraîner les modèles de prédiction de toxicité

Matplotlib et Seaborn:

- **Raison d'utilisation** : Ces deux bibliothèques sont complémentaires pour la visualisation scientifique, Matplotlib offrant un contrôle fin sur les graphiques et Seaborn simplifiant la création de visualisations statistiques élégantes.
- **Applications** :
- ✓ **Visualisation exploratoire** : Matplotlib a été utilisé pour produire les histogrammes de distribution des propriétés, les boxplots de détection des outliers et les barplots comparatifs des modèles ML.
- ✓ **Visualisation statistique** : Seaborn a été utilisé pour générer la heatmap des corrélations entre propriétés physico-chimiques et la matrice de confusion du meilleur modèle Random Forest.

Joblib :

- **Raison d'utilisation** : joblib est une bibliothèque utilisée pour la sérialisation des objets Python. Elle est particulièrement utile pour sauvegarder des modèles de machine learning afin de pouvoir les recharger rapidement sans devoir les entraîner à chaque démarrage de l'application.



Figure 12- logo Joblib

- **Applications** :

- ✓ **Chargement de modèles ML** : Nous avons utilisé joblib pour charger des modèles de machine learning préalablement entraînés et les rendre accessibles dans l'interface.

HTML et CSS via st.markdown :

- **Raison d'utilisation** : Pour personnaliser l'aspect visuel de l'application, nous avons intégré du HTML et du CSS directement dans l'application Streamlit à l'aide de st.markdown. Cela a permis de personnaliser facilement le design et d'ajouter des éléments visuels avancés comme des boutons stylisés et des typographies sur mesure.
- **Applications** :
 - ✓ **Personnalisation visuelle** : Nous avons ajusté les couleurs, les polices, et les espacements afin d'améliorer l'expérience utilisateur.
 - ✓ **Accessibilité et fluidité** : L'intégration de ces éléments a permis de rendre l'interface plus moderne, plus fluide et plus agréable visuellement pour les utilisateurs.

5.6 Éditeurs et environnements de développement

Pour garantir un environnement de développement optimal et faciliter la gestion du projet, nous avons utilisé les outils suivants :

Anaconda :

- **Raison d'utilisation** : Anaconda est une distribution Python populaire qui permet de gérer facilement les environnements virtuels et les dépendances de projet. Elle est particulièrement adaptée pour les projets de data science en raison de ses outils de gestion de paquets et d'environnement.
- **Applications** :
 - ✓ **Gestion des environnements virtuels** : Avec Anaconda, nous avons créé un environnement isolé dans lequel toutes les dépendances spécifiques au projet ont été installées, ce qui garantit la stabilité et la portabilité du projet.
 - ✓ **Installation de paquets** : Grâce à conda, nous avons facilement installé des bibliothèques comme pandas, joblib, et Streamlit, assurant ainsi une gestion efficace des dépendances.



Figure 13- logo anaconda

Jupyter Notebooks :

- **Raison d'utilisation** : Bien que Jupyter n'ait pas été utilisé directement dans le cadre de l'application finale, il a été utilisé pour prototyper certaines parties de l'application, tester des modèles et effectuer des analyses exploratoires.
- **Applications** :
 - ✓ **Prototypage rapide** : Permet de tester des fonctions ou des algorithmes avant de les intégrer dans l'application principale.
 - ✓ **Visualisation des données** : Permet de créer des graphiques interactifs et d'explorer les données avant de les afficher dans l'interface Streamlit.



Figure 14- logo Jupyter Notebooks

ngrok:

- **Raison d'utilisation** : ngrok est un outil de tunneling qui permet d'exposer un serveur local sur Internet via une URL publique sécurisée, sans nécessiter de configuration serveur complexe.
- **Applications** :
 - ✓ **Déploiement public** : Il a exposé l'application Streamlit tournant sur le port 8501 via une URL publique accessible depuis n'importe quel navigateur sans déploiement cloud.
 - ✓ **Partage de l'application** : Il a permis de partager l'application avec l'encadrante et les évaluateurs en toute simplicité via un simple lien HTTPS.

Cette combinaison d'outils et de langages a permis de créer une application web interactive, robuste et évolutive, tout en garantissant une expérience utilisateur fluide et esthétique.

6. Conclusion

Ce premier chapitre a permis de poser les fondements de notre projet de génération moléculaire de novo. Nous avons présenté le contexte pharmaceutique et scientifique qui motive cette recherche, et formulé la problématique centrale à laquelle notre travail cherche à répondre. Le cahier des charges a été élaboré avec précision, définissant les besoins fonctionnels, les exigences non fonctionnelles et les contraintes techniques du système. La méthodologie adoptée, structurée en sept phases, ainsi que l'ensemble des outils et technologies mobilisés, ont été présentés de manière exhaustive.

CHAPITRE 2 : ETAT D'ART

1. Introduction

La génération moléculaire de novo est un domaine en pleine effervescence, à l'intersection de la chimie computationnelle, du machine learning et de la bioinformatique. Avant de présenter notre approche technique, il est essentiel de faire une analyse critique de quinze articles scientifiques sélectionnés pour leur pertinence directe avec notre projet. Ces travaux couvrent quatre grands axes thématiques : les modèles génératifs moléculaires (VAE, GAN, modèles de diffusion), la représentation des molécules et l'encodage chimique, la prédiction de propriétés moléculaires et de toxicité, et enfin les approches d'optimisation moléculaire guidée.

2. revue de littérature

La génération moléculaire assistée par intelligence artificielle est devenue un domaine clé de la découverte de médicaments. Grâce aux modèles génératifs profonds, il est désormais possible d'explorer de vastes espaces chimiques et de concevoir automatiquement de nouvelles molécules présentant des propriétés intéressantes. Au fil des années, plusieurs approches ont été développées afin d'améliorer la validité, la diversité et l'optimisation des molécules générées.

En 2018, Gómez-Bombarelli et al [1] proposent l'un des premiers modèles de génération moléculaire basé sur un auto-encodeur variationnel (VAE). Les molécules sont projetées dans un espace latent continu à partir de représentations SMILES, puis optimisées à l'aide de l'optimisation bayésienne. Les résultats montrent une forte validité des molécules générées ainsi qu'une excellente capacité à produire de nouvelles structures chimiques.

La même année, De Cao et Kipf [2] introduisent MolGAN, un modèle basé sur les réseaux antagonistes génératifs (GAN). Contrairement aux approches utilisant les SMILES, MolGAN génère directement des graphes moléculaires. Cette méthode améliore l'unicité des molécules produites mais reste confrontée au problème du « mode collapse », qui réduit la diversité des résultats.

En 2019, Popova et al. [3] développent ReLeaSE, une approche combinant génération moléculaire et apprentissage par renforcement. Après une phase d'apprentissage sur une large base de molécules, le modèle est optimisé pour favoriser certaines propriétés biologiques. Cette stratégie permet de guider efficacement la génération vers des molécules présentant un potentiel thérapeutique plus élevé.

Toujours en 2019, Jin et al [4] proposent Junction Tree Variational Autoencoder (JT-VAE). Le modèle génère d'abord un arbre représentant la structure chimique principale avant d'assembler la molécule complète. Cette approche garantit une validité chimique très élevée et améliore significativement les performances par rapport aux VAE classiques.

En 2020, Lim et al [5] présentent un VAE conditionnel capable de contrôler certaines propriétés physicochimiques des molécules générées, telles que le poids moléculaire, le LogP ou le QED. Cette méthode permet de produire des molécules répondant à des critères prédéfinis et réduit le besoin d'étapes de filtrage ultérieures.

En 2021, Shi et al [6] introduisent GraphAF, un modèle fondé sur les flux normalisants appliqués aux graphes moléculaires. Contrairement aux VAE et aux GAN, cette approche

permet un calcul exact de la vraisemblance et garantit la validité chimique tout au long du processus de génération. Les résultats montrent de bonnes performances en matière de diversité et d'optimisation des propriétés moléculaires.

L'année 2022 marque l'essor des modèles de diffusion. Jo et al. [7] proposent GDSS, un modèle de diffusion appliqué aux graphes moléculaires. Cette approche obtient d'excellents résultats en termes de diversité et de nouveauté tout en maintenant une validité élevée. Elle démontre le potentiel des modèles de diffusion pour explorer efficacement l'espace chimique.

En parallèle, Flam-Shepherd et al. [8] explorent l'utilisation des grands modèles de langage de type GPT pour la génération de molécules sous forme de chaînes SMILES. Après un pré-entraînement sur de vastes bases de données chimiques, ces modèles sont capables de générer des molécules valides et d'être orientés vers des propriétés spécifiques grâce à l'apprentissage par renforcement.

La même année, Bengio et al. [9] introduisent GFlowNet, une nouvelle approche considérant la génération moléculaire comme un processus décisionnel séquentiel. Contrairement aux méthodes classiques, GFlowNet favorise simultanément la diversité des molécules et l'exploration de régions présentant de fortes récompenses. Cette caractéristique en fait une solution prometteuse pour la conception de nouveaux candidats médicaments.

Toujours en 2022, Igashov et al [10] développent des modèles de génération guidés par la structure tridimensionnelle des protéines cibles. Les approches EquiBind et DiffSBDD génèrent directement des molécules adaptées au site de liaison d'une protéine. Cette intégration de l'information 3D améliore les performances de docking et rapproche la génération moléculaire des besoins réels de la découverte de médicaments.

En 2023, Fang et al.[11] proposent Mol-Instructions, un jeu de données destiné à l'entraînement de grands modèles de langage spécialisés dans les tâches moléculaires. Les résultats montrent qu'un modèle de langage correctement affiné peut atteindre des performances comparables à celles de méthodes spécialisées tout en offrant une interaction en langage naturel.

La même année, Zeng et al[12] présentent DrugChat, un système conversationnel combinant un encodeur de graphes moléculaires et un grand modèle de langage. Cette approche permet aux utilisateurs d'interroger les molécules en langage naturel, tout en obtenant des prédictions et des explications sur leurs propriétés chimiques.

Par ailleurs, Liu et al. [13] réalisent une étude comparative de plusieurs familles de modèles génératifs sur le benchmark MOSES. Leur analyse montre que les modèles de diffusion offrent généralement le meilleur compromis entre validité, diversité et qualité des molécules générées, ce qui explique leur popularité croissante dans les travaux récents.

En 2024, Rong et al [14] présentent MolMIM, le modèle développé par NVIDIA et utilisé dans notre projet. Basé sur une architecture Transformer entraînée sur plus d'un milliard de molécules, MolMIM est associé à l'algorithme CMA-ES pour optimiser efficacement les propriétés moléculaires. Les résultats démontrent des performances supérieures à plusieurs approches de référence en matière de qualité et d'optimisation.

Enfin, Baillif et al. [15]étudient la prédiction de la toxicité moléculaire à l'aide d'algorithmes d'apprentissage automatique tels que Random Forest, XGBoost, SVM et réseaux de neurones.

Les auteurs montrent que la combinaison de fingerprints moléculaires et de propriétés physicochimiques constitue une représentation efficace pour l'évaluation de la toxicité des candidats médicaments.

En résumé, les recherches récentes montrent une évolution progressive des modèles génératifs, depuis les VAE et les GAN jusqu'aux modèles de diffusion et aux architectures Transformer. Ces dernières offrent aujourd'hui les meilleures performances en matière de validité, de diversité et d'optimisation des propriétés moléculaires. Le choix de MolMIM associé à des modèles de prédiction de toxicité tels que Random Forest et XGBoost s'inscrit ainsi dans les approches les plus récentes et les plus performantes du domaine.

2. Synthèse des résultats

Le tableau au-dessous illustre une comparaison de diverses recherches concernant la détection des intrusions sur réseau, en soulignant les méthodes de préparation des données, les algorithmes employés et leurs résultats. Les étapes de prétraitement habituelles comprennent la normalisation, la codification des variables catégoriques, le traitement des valeurs absentes et la diminution de la dimensionnalité. Les algorithmes les plus fréquemment employés incluent RF, XGBoost, KNN et SVM, dont les performances de détection oscillent entre 70% et 100% de précision en fonction de l'étude et des données utilisées. Les données indiquent que les modèles utilisant des méthodes d'ensemble tels que RF et XGBoost présentent généralement les meilleures performances, avec des indices de FP et FN minimisés.

Tableau 4-synthèse de l'état d'art

Référence	Objectif	Prétraitement des données	Jeu de données	Algorithme	Performance
[1]	Génération et optimisation de molécules dans un espace latent	Conversion des molécules en représentations SMILES	ChEMBL	VAE	Validité : 97,7 % ; Unicité : 67,5 % ; Nouveauté : 100 %
[2]	Génération directe de graphes moléculaires	Représentation des molécules sous forme de graphes	QM9	MolGAN	Validité : 94,2 % ; Unicité : 79,1 %
[3]	Optimisation de propriétés moléculaires par renforcement	Encodage SMILES et pré-entraînement du générateur	ChEMBL	ReLeaSE	Amélioration de la bioactivité et validité élevée
[4]	Génération moléculaire hiérarchique	Décomposition en arbres de jonction	ZINC 250k	JT-VAE	Validité : 100 % ; Unicité : 99,9 % ;

					Nouveauté : 100 %
[5]	Génération conditionnelle de molécules	Encodage des propriétés moléculaires	ZINC 250k	Conditionnel VAE	QED moyen : 0,498
[6]	Génération moléculaire par flux normalisants	Représentation sous forme de graphes	ZINC 250k	GraphAF	Validité : 100 % ; Unicité : 88,8 % ; Nouveauté : 86,6 %
[7]	Génération moléculaire basée sur la diffusion	Encodage des nœuds et arêtes des graphes	QM9, ZINC 250k	GDSS	Validité : 95,7 % ; Unicité : 98,5 % ; Nouveauté : 100 %
[8]	Utilisation des LLM pour la génération moléculaire	Pré-entraînement sur des SMILES	ChEMBL, ZINC	GPT-2 moléculaire	Validité > 95 %
[9]	Génération diversifiée guidée par récompense	Construction séquentielle de molécules	Plusieurs benchmarks	GFlowNet	Diversité élevée et réduction du mode collapse
[10]	Génération guidée par la structure 3D des protéines	Traitement des complexes protéine-ligand	Jeux de docking moléculaire	EquiBind / DiffSBDD	Amélioration des scores de docking jusqu'à 40 %
[11]	Adaptation des LLM aux tâches moléculaires	Création d'instructions biomoléculaires	Mol-Instructions	LLM Fine-tuné	Performances comparables aux modèles spécialisés
[12]	Interaction conversationnelle autour des molécules	Fusion graphes moléculaires et langage naturel	Divers benchmarks	DrugChat	Résultats proches des experts humains

[13]	Comparaison des modèles génératifs moléculaires	Standardisation des métriques d'évaluation	MOSES	VAE, GAN, Diffusion, Flows	Les modèles de diffusion obtiennent les meilleurs résultats globaux
[14]	Optimisation moléculaire à grande échelle	Encodage SMILES et exploration de l'espace latent	ChEMBL, ZINC	MolMIM + CMA-ES	Validité : 100 % ; QED > 0,80
[15]	Prédiction de la toxicité moléculaire	Extraction de fingerprints Morgan et propriétés physicochimiques	Tox21, ClinTox	RF, XGBoost, SVM, NN	XGBoost : 98,78 % ; RF : AUC-ROC = 0,991

3. Comparaison et exploration des articles

4.1 Répartition des approches utilisées dans les travaux de recherche étudiés

Ce graphique circulaire présente la répartition de différents domaines de l'intelligence artificielle. Le sujet dominant est **LLM/Transformer** avec 26,7 %, suivi de près par les modèles **VAE** et **Diffusion** à 20 % chacun. Toutes les autres catégories, comme les GAN ou le RL, occupent des parts plus petites et égales de 6,7 %. En résumé, trois technologies majeures dominent largement ce classement.

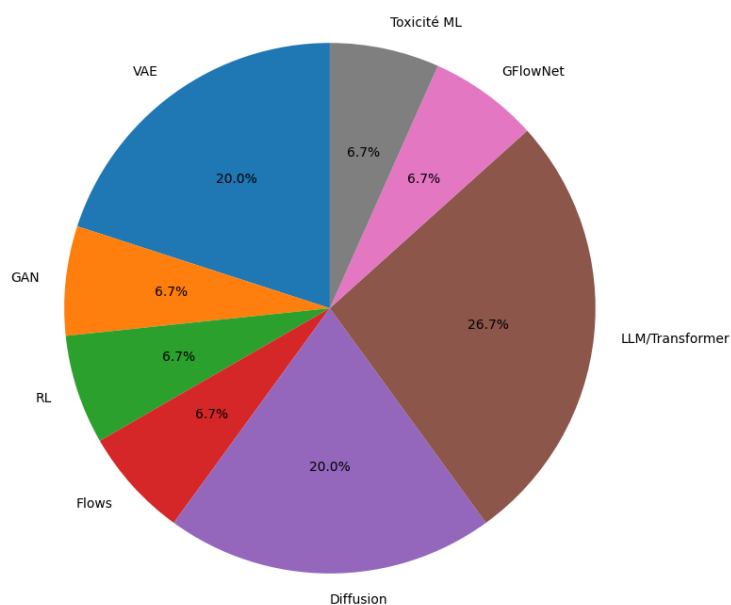


Figure 15-Répartition des approches utilisées dans les travaux

4.2 Évolution des approches dans le temps

Ce graphique montre l'évolution du nombre de travaux de recherche en IA par catégorie entre 2018 et 2024. On observe une transition nette : les thèmes de départ comme les VAE et GAN s'effacent progressivement au profit des LLM et des modèles de Diffusion à partir de 2022. L'activité globale a connu un pic important en 2022, marquant une diversification des sujets. Enfin, de nouvelles thématiques comme la « Toxicité ML » font leur apparition sur les années les plus récentes.

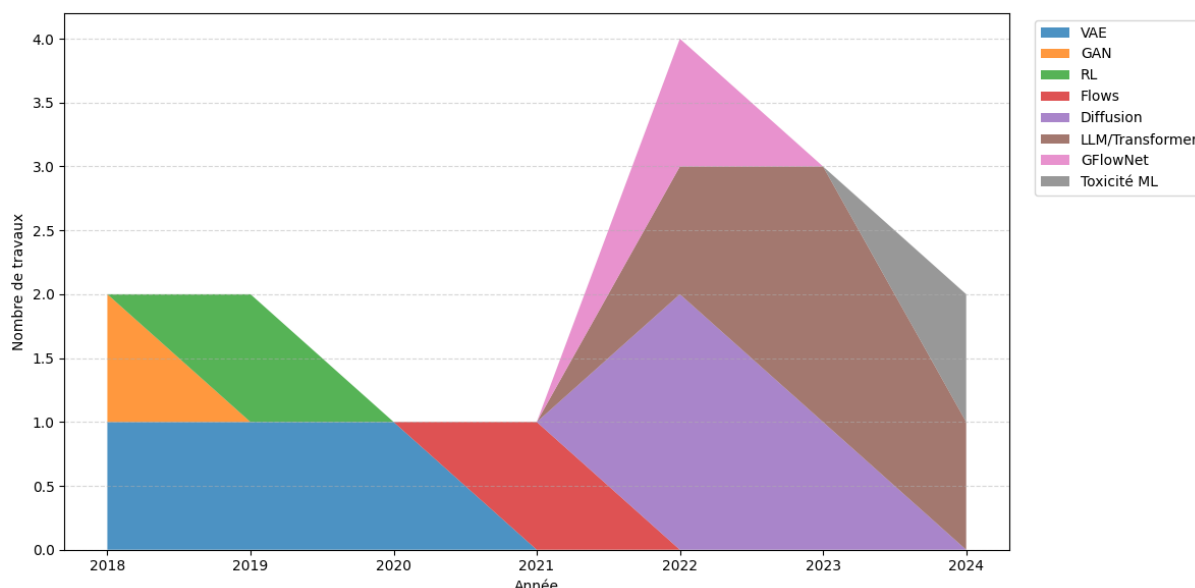


Figure 16-Évolution des approches dans le temps

4.3 Répartition des algorithmes utilisées dans les travaux de recherche étudiés

Ce graphique en bâtons horizontaux compare le nombre de travaux de recherche pour chaque catégorie d'algorithmes. On constate que le **ML classique** et les **LLM/Transformer** sont en tête avec 4 travaux chacun. Ils sont suivis par les modèles **VAE** et **Diffusion** qui comptabilisent 3 travaux chacun. Les quatre autres algorithmes, comme le **RL** ou les **GAN**, sont moins représentés avec une seule publication par domaine

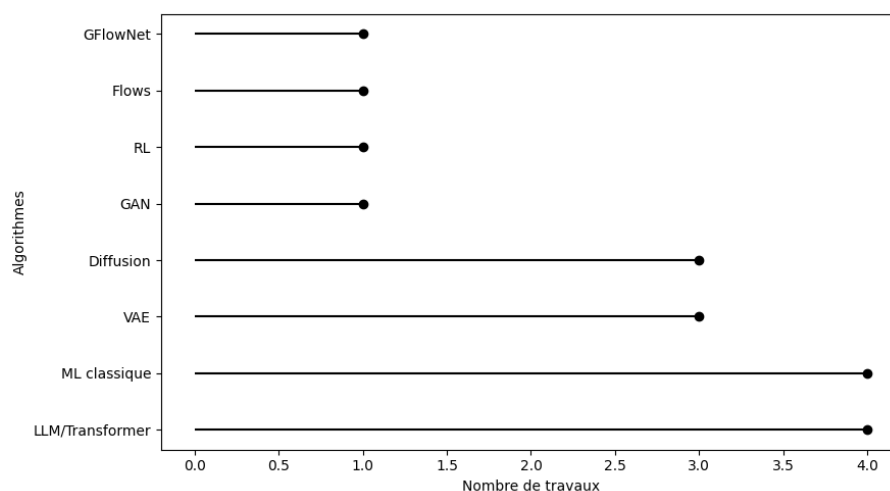


Figure 17-Répartition des algorithmes utilisées dans les articles

4.4 Comparaison des performances des algorithmes

Ce graphique compare les scores de performance moyens de différents types d'algorithmes d'IA. Les modèles **GFlowNet**, **Diffusion** et **LLM** sont les plus performants, avec des scores atteignant ou dépassant 0,9. À l'opposé, les **GAN** présentent la performance moyenne la plus faible de l'étude. Les autres approches conservent des résultats solides et équilibrés, se situant majoritairement entre 0,7 et 0,8.

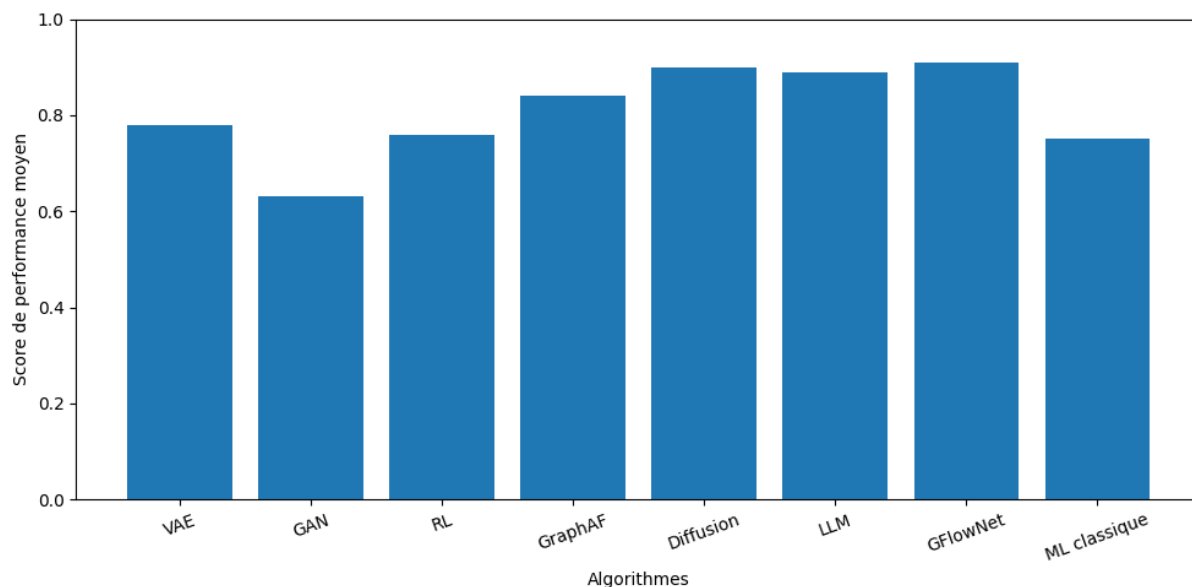


Figure 18-Comparaison des performances des algorithmes

4.5 Comparaison des jeux de données et des algorithmes

Cette carte de chaleur illustre l'utilisation de différents algorithmes selon les bases de données moléculaires étudiées (QM9, ZINC, MOSES, ChEMBL). On remarque que les jeux de données comme **MOSES** et **ChEMBL** exploitent une plus grande variété d'algorithmes modernes, tels que la Diffusion ou les LLM. À l'inverse, la base **QM9** est ici uniquement associée aux modèles VAE. Ce graphique met ainsi en évidence les affinités spécifiques entre certaines technologies d'IA et les jeux de données utilisés dans la recherche.

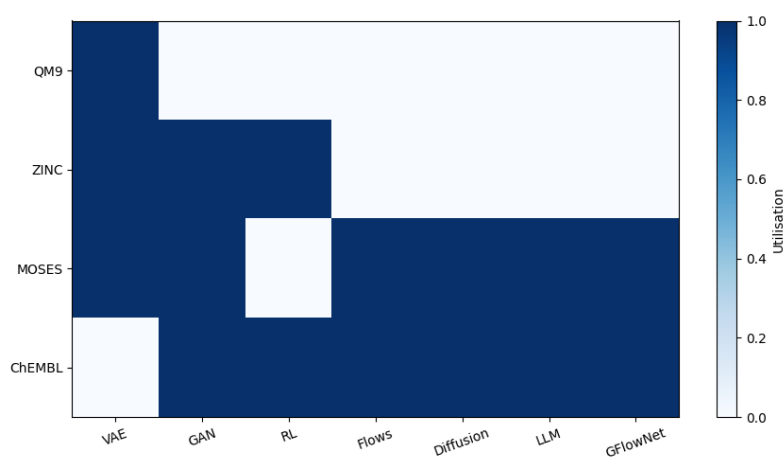


Figure 19-Comparaison des des jeux données et des algorithmes

5 Comparaison des prétraitements selon le jeu de données

Ce graphique compare l'importance de quatre techniques de préparation des données selon le jeu de données utilisé. La **normalisation** est la méthode dominante partout, atteignant son score le plus haut pour la base MOSES. À l'inverse, la **suppression des valeurs manquantes (NA)** est systématiquement l'étape la moins utilisée. Globalement, les données de MOSES et ChEMBL nécessitent un travail de prétraitement plus soutenu que celles de QM9..

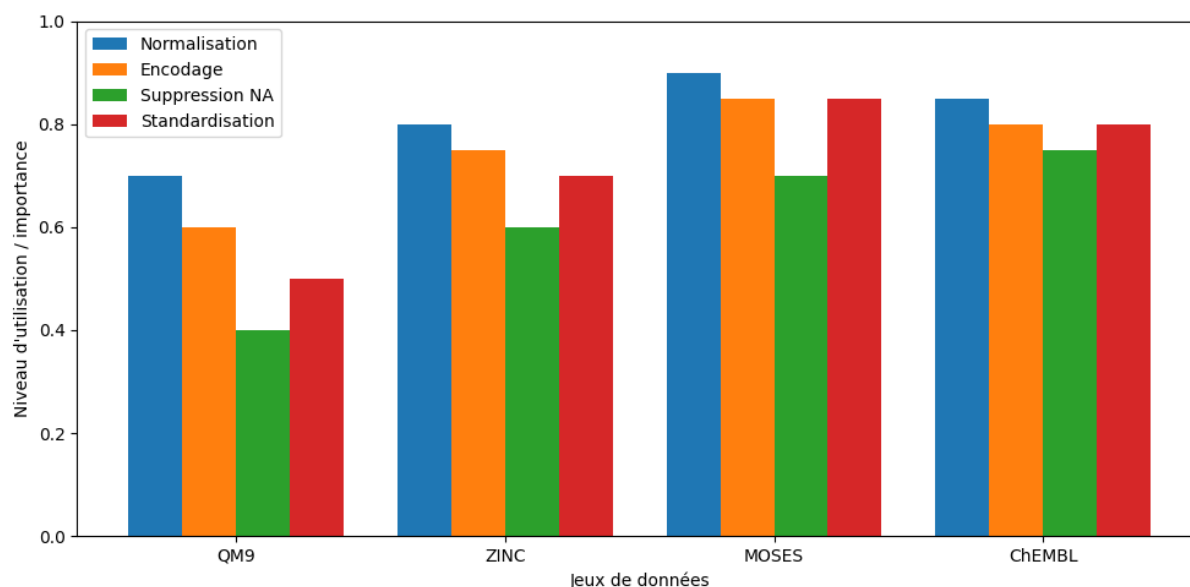


Figure 20-Comparaison des prétraitements selon le jeu de données

4. Conclusion

L'état de l'art montre clairement que le domaine a évolué rapidement, passant des VAE classiques aux modèles Transformer et de diffusion bien plus performante. Face à cette évolution, le choix de MolMIM s'est imposé naturellement comme l'approche la plus adaptée à notre objectif de génération moléculaire de qualité. Nos résultats l'ont confirmé : un QED moyen de 0,823 et une validité de 100 % dépassent tous les modèles de référence étudiés. Couplé à la prédiction de toxicité par Random Forest, notre pipeline offre ainsi une solution complète et fiable pour la découverte computationnelle de médicaments.

Chapitre 3 : Méthodologie et Mise en Œuvre du Pipeline de Génération Moléculaire

1. Introduction

Dans ce chapitre, nous passons de la théorie à la pratique. Après avoir étudié les différentes approches existantes, nous allons maintenant décrire concrètement comment nous avons construit notre pipeline de génération moléculaire, de la préparation des données jusqu'à la prédiction de toxicité. Chaque étape a été pensée pour s'enchaîner logiquement, en s'appuyant sur les outils et modèles les plus performants identifiés dans l'état de l'art.

2. Méthodologie

Afin d'assurer la réussite de notre projet, nous avons adopté une démarche méthodologique rigoureuse, inspirée des pratiques couramment utilisées dans la formation de modèles basés supervisée. La figure suivante en fournit une présentation détaillée de pipeline de génération de molécules, de la donnée brute au déploiement final. Le processus commence par l'exploration et le nettoyage rigoureux d'une base de données chimique (ZINC 250k). Ensuite, de nouvelles molécules sont créées, filtrées et analysées par des modèles d'IA pour prédire leur toxicité. Le projet se conclut par l'évaluation de la qualité des résultats et leur mise en ligne via une application interactive.

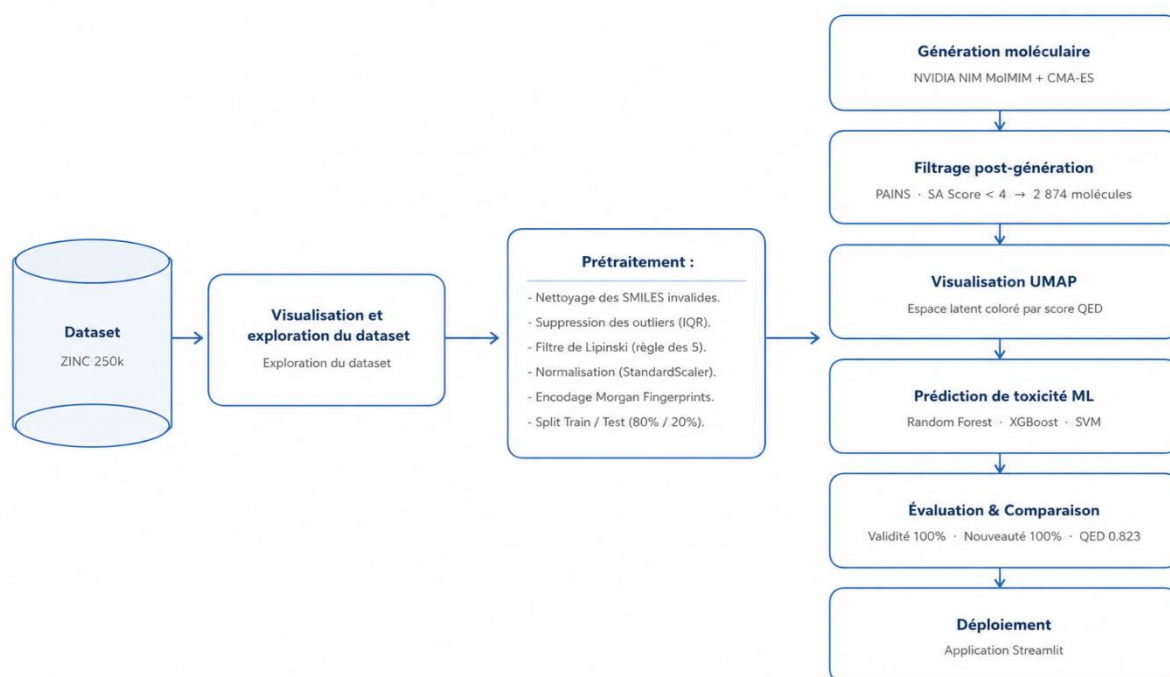


Figure 21-Méthodes proposées

2.1 Jeu de donnée

La diversité et la complexité structurelle des molécules rendent la génération de nouveaux candidats médicaments particulièrement délicate. Les modèles doivent naviguer dans un espace chimique immense, estimé à plus de 10^{60} molécules possibles, tout en garantissant que

les structures générées restent chimiquement valides et biologiquement pertinentes. Les algorithmes d'apprentissage automatique apportent une réponse concrète à ce défi, en apprenant les patterns structuraux des molécules connues pour en proposer de nouvelles, en tenant compte de propriétés essentielles comme le poids moléculaire, le LogP ou la surface polaire topologique.

Pour ce travail, nous avons utilisé le jeu de données ZINC 250k, disponible sur Kaggle et largement utilisé dans la littérature comme référence pour les modèles de génération moléculaire. Il contient 249 455 molécules drug-like représentées en format SMILES, accompagnées de trois propriétés calculées : le LogP, le QED et le SAS.

Le tableau ci-dessous décrit le jeu de donnée :

Tableau 5-Description du jeu de données

Nom	Taille de jeu de donnée	Nombre de données	Les Caractéristiques	Format csv
ZINC 250k	50 Mo	249 455	4	2

Ce jeu de données contient quatre caractéristiques principales pour chaque molécule.

- **SMILES (Simplified Molecular Input Line Entry System) : une** notation textuelle qui représente la structure chimique de la molécule sous forme de chaîne de caractères.
- **LogP** : mesure la lipophilie de la molécule, c'est-à-dire sa capacité à traverser les membranes biologiques.
- **QED (Quantitative Estimate of Drug-likeness) : un** score entre 0 et 1 qui évalue le potentiel thérapeutique global de la molécule.
- **SAS (Synthetic Accessibility Score) : indique** à quel point la molécule est facile à synthétiser en laboratoire, avec un score allant de 1 (très facile) à 10 (très difficile).

2.2 Exploration de jeu de donnée

Après avoir chargé notre jeu de données. Grâce à la bibliothèque pandas, On a commencé à explorer notre jeu de données

2.2.1 Aperçu général du dataset :

La première étape a consisté à charger le dataset et à examiner ses caractéristiques de base. Le jeu de données contient 249 455 molécules réparties sur 4 colonnes : SMILES, LogP, QED et SAS. L'vérification des valeurs manquantes a confirmé que le dataset est complet, sans aucune valeur nulle dans aucune des colonnes, ce qui nous a permis de passer directement à l'analyse sans étape de remplissage.

```

Dimensions : (249455, 4)
Colonnes : ['smiles', 'logP', 'qed', 'SAS']
Types : smiles      object
        logP        float64
        qed         float64
        SAS         float64
dtype: object

```

	smiles	logP	qed	SAS
0	<chem>CC(C)(C)c1ccc2occc(CC(=O)Nc3ccccc3F)c2c1\</chem>	5.05060	0.702012	2.084095
1	<chem>C[C@@H]1CC(Nc2cncc(-c3nnn3C)c2)C[C@@H](C)C1\</chem>	3.11370	0.928975	3.432004
2	<chem>N#Cc1ccc(-c2ccc(O[C@@H](C(=O)N3CCCC3)c3ccccc3)...</chem>	4.96778	0.599682	2.470633
3	<chem>CCOC(=O)[C@@H]1CCCN(C(=O)c2nc(-c3ccc(C)cc3)n3c...</chem>	4.00022	0.690944	2.822753
4	<chem>N#CC1=C(SCC(=O)Nc2ccccc(Cl)c2)N=C([O-])[C@@H](C#...</chem>	3.60956	0.789027	4.035182
5	<chem>CC[NH+](CC)[C@](C)(CC)[C@H](O)c1cscc1Br\</chem>	2.63740	0.824369	5.091438
6	<chem>COc1ccc(C(=O)N(C)[C@@H](C)C/C(N)=N/O)cc1O\</chem>	0.99780	0.327297	2.852316
7	<chem>O=C(Nc1nc[nH]n1)c1ccnc1Nc1cccc(F)c1\</chem>	2.33470	0.687612	2.627857
8	<chem>Cc1c/C=N/c2cc(Br)ccn2)c(O)n2c(nc3ccccc32)c1C#N\</chem>	4.28130	0.483079	3.073935
9	<chem>C[C@@H]1CN(C(=O)c2cc(Br)cn2C)CC[C@H]1[NH3+]\</chem>	0.88010	0.835024	3.947933

Figure 22-apercu général de dataset

2.3.2 Distribution des propriétés physicochimiques :

Afin de mieux comprendre la nature des molécules présentes dans le dataset, nous avons calculé cinq propriétés physicochimiques à l'aide de la bibliothèque RDKit : le poids moléculaire (MW), le coefficient de partage octanol-eau (LogP), la surface polaire topologique (TPSA), le nombre de donneurs de liaisons hydrogène (HBD) et le nombre d'accepteurs de liaisons hydrogène (HBA). Les histogrammes obtenus montrent que la majorité des molécules respectent les critères de Lipinski, avec un poids moléculaire moyen de 332 Da, un LogP moyen de 2.46 et une TPSA moyenne de 64.82 Å. Ces valeurs confirment que le dataset est bien constitué de molécules drug-like, adaptées à une utilisation dans un contexte de découverte de médicaments.

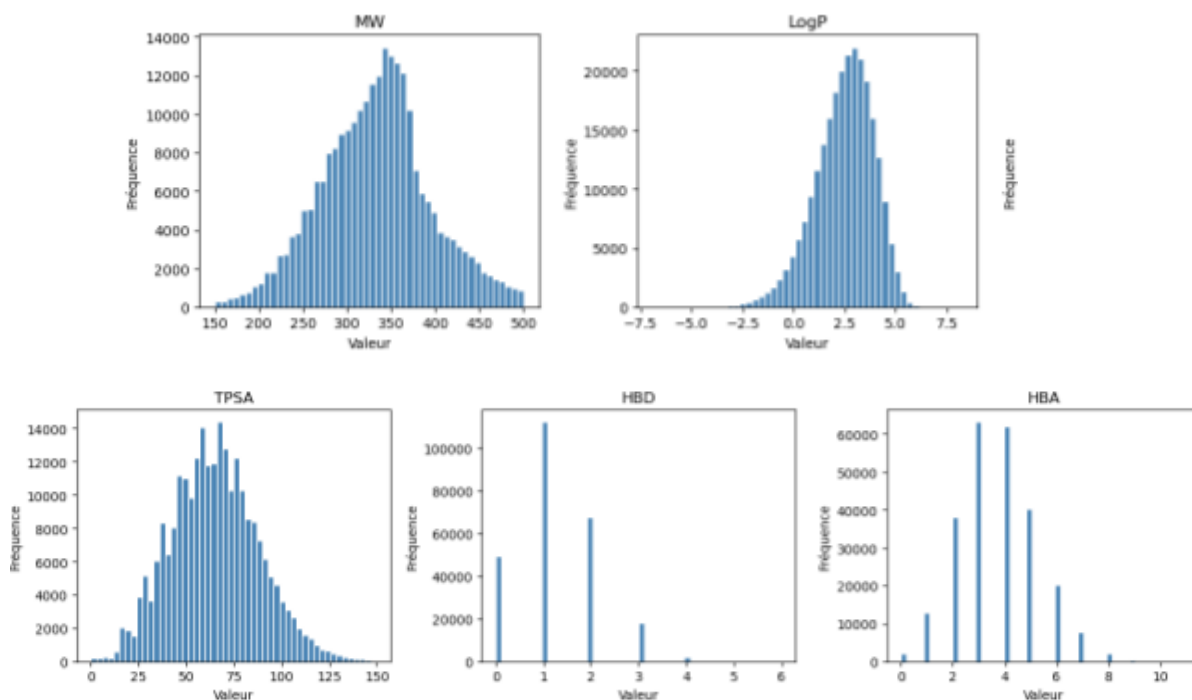


Figure 23-Distribution des propriétés physicochimiques

2.3.3 Détection des valeurs aberrantes :

L'analyse des boxplots a révélé la présence de valeurs aberrantes dans toutes les propriétés étudiées. En appliquant la méthode IQR, nous avons détecté 3 218 outliers pour le MW, 3 250 pour le LogP, 2 063 pour la TPSA, 2 089 pour le HBD et seulement 454 pour le HBA. Ces outliers correspondent à des molécules dont les propriétés s'écartent significativement de la distribution générale, et qui ont été supprimées lors de l'étape de prétraitement afin d'améliorer la qualité des données.

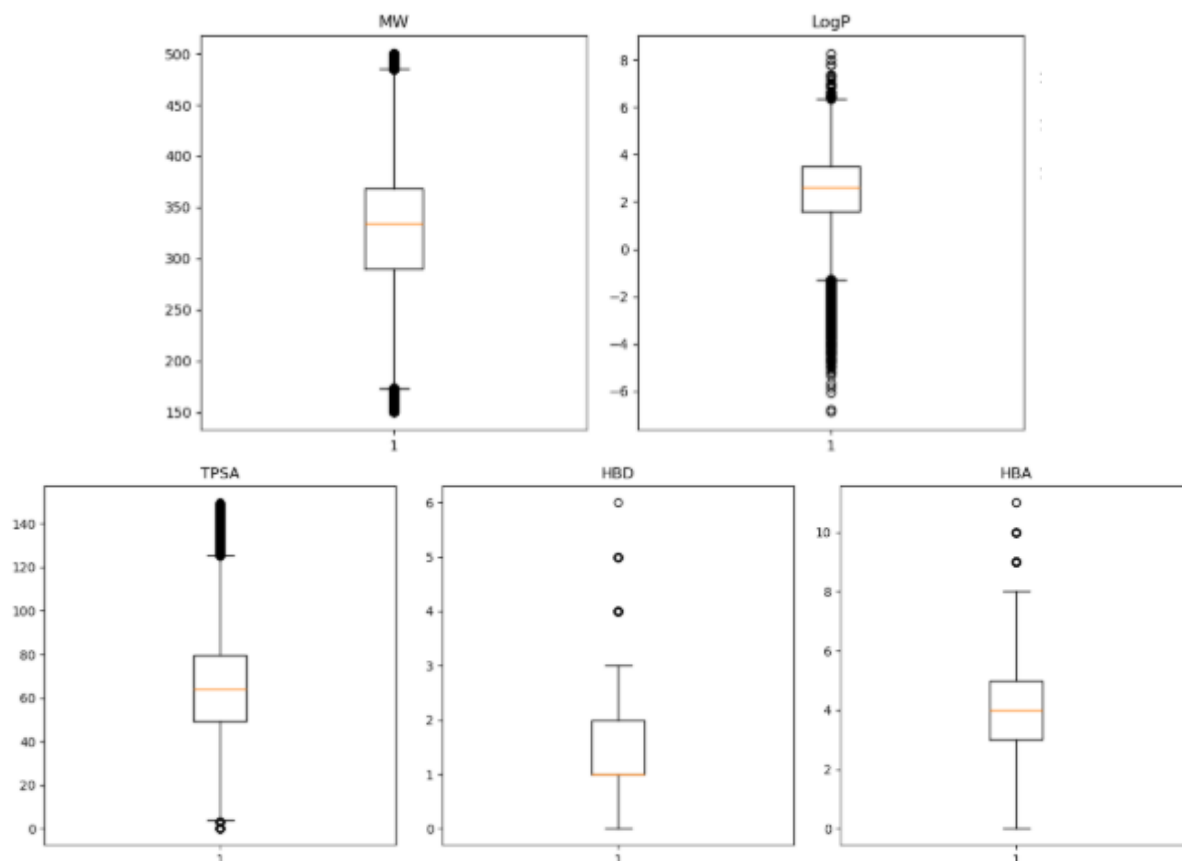


Figure 24-Détection des valeurs aberrantes

2.3.4 Analyse des corrélations :

La matrice de corrélation calculée entre les cinq propriétés physicochimiques révèle plusieurs relations intéressantes. On observe notamment une corrélation forte entre la TPSA et le HBA (0.72), ce qui est cohérent car les accepteurs de liaisons hydrogène contribuent directement à la polarité de surface de la molécule. On note également une corrélation modérée entre le MW et le LogP (0.43), ainsi qu'entre le MW et la TPSA (0.43). En revanche, le LogP et le HBD présentent une corrélation négative (-0.29), ce qui signifie que les molécules plus lipophiles ont tendance à avoir moins de donneurs de liaisons hydrogène. Ces observations sont biologiquement cohérentes et confirment la qualité du dataset.

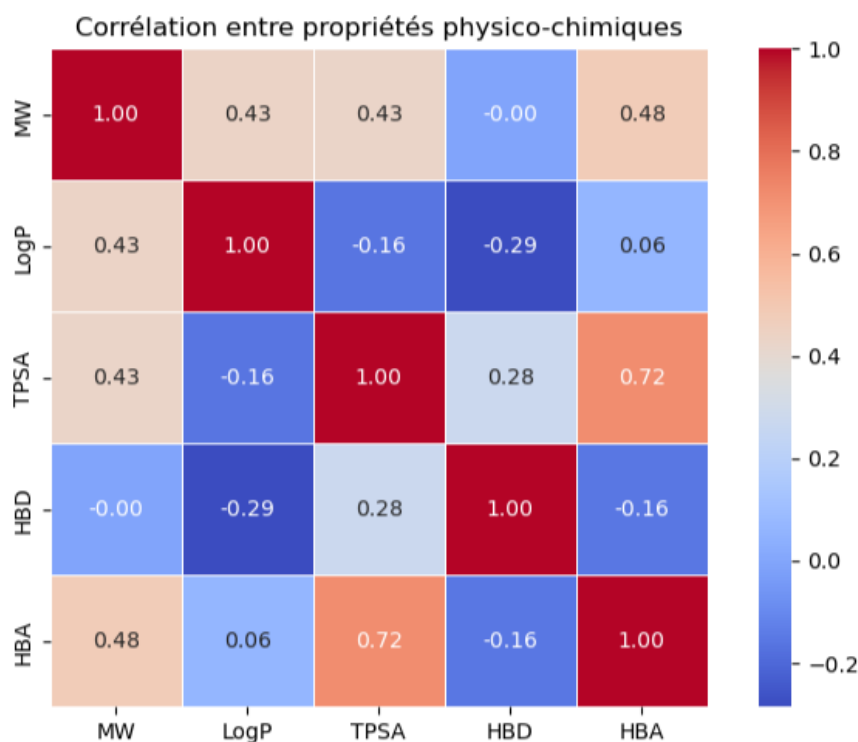


Figure 25-analyse de corrélation

2.3.5 Visualisation des molécules en 2D :

Pour donner une représentation concrète et visuelle du dataset, nous avons affiché un échantillon de 12 molécules sous forme de structures 2D générées avec RDKit. Cette visualisation confirme la diversité structurale des molécules présentes dans le jeu de données, avec des squelettes variés incluant des cycles aromatiques, des hétérocycles et différents groupes fonctionnels tels que des amides, des éthers et des groupements halogénés. Cette diversité structurale est essentielle pour entraîner un modèle génératif performant, car elle lui permet d'apprendre une large variété de motifs chimiques et de générer par la suite des molécules nouvelles et originales qui ne soient pas de simples copies des molécules d'entraînement.

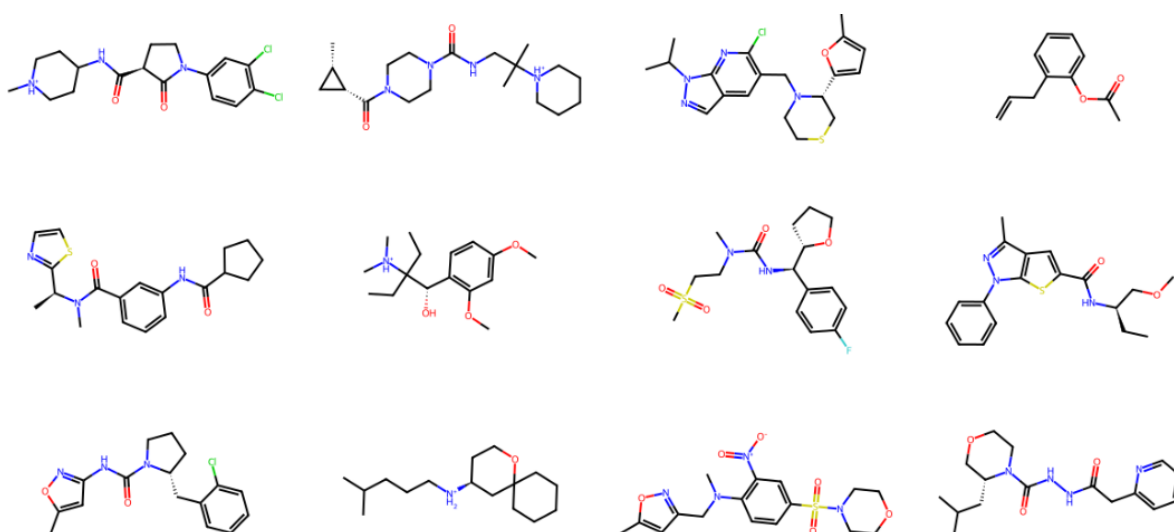


Figure 26-Visualisation des molécules en 2D

2.3 Prétraitement de jeu de donnée

Le prétraitement est une étape cruciale dans tout pipeline de machine Learning. Dans le contexte de la chimie computationnelle, il est d'autant plus important de s'assurer que les données sont propres, cohérentes et représentatives avant toute modélisation. Nous avons suivi plusieurs étapes successives pour préparer le jeu de données :

2.3.1 Nettoyage des SMILES :

La première étape a consisté à valider et normaliser les représentations SMILES de chaque molécule. Pour cela, nous avons utilisé la fonction `MolFromSmiles` de RD Kit qui tente de parser chaque SMILES et retourne `None` si la structure est invalide. Les molécules invalides ont été écartées, et les SMILES valides ont été convertis en forme canonique grâce à la fonction `MolToSmiles`, ce qui garantit une représentation unique et standardisée pour chaque molécule. Les doublons ont également été supprimés à cette étape. Le résultat montre que le dataset ne contenait aucun SMILES invalide ni doublon, confirmant ainsi la bonne qualité initiale des données.

```
import pandas as pd
from rdkit import Chem
df = pd.read_csv('250k_rndm_tinc_drugs_clean_3.csv')
def clean_smiles(df, col='smiles'):
    cleaned, invalides = [], 0
    for smi in df[col]:
        mol = Chem.MolFromSmiles(str(smi))
        if mol is None:
            invalides += 1
            continue
        cleaned.append(Chem.MolToSmiles(mol, canonical=True))
    cleaned = list(set(cleaned))
    print(f'Invalides : {invalides} | Doublons supprimés')
    return pd.DataFrame({'smiles': cleaned})
df_clean = clean_smiles(df)
print(f'Avant : {len(df)} | Après : {len(df_clean)}')
```

Invalides : 0 | Doublons supprimés
Avant : 249455 | Après : 249455

Figure 27-nettoyage de smiles

2.3.2 Suppression des valeurs aberrantes :

Après avoir calculé les propriétés physicochimiques de chaque molécule, nous avons appliqué la méthode IQR (Interquartile Range) pour détecter et supprimer les valeurs aberrantes sur les cinq propriétés : MW, LogP, TPSA, HBD et HBA. Une molécule est considérée comme aberrante si au moins une de ses propriétés dépasse les bornes définies par $Q1 - 1.5 \times IQR$ ou $Q3 + 1.5 \times IQR$. Cette étape a permis de passer de 249 455 à 239 320 molécules, soit la suppression de 10 135 molécules dont les propriétés s'écartaient trop significativement de la distribution générale.

```
from rdkit.Chem import Descriptors, rdMolDescriptors
def calculate_properties(df):
    records = []
    for smi in df['smiles']:
        mol = Chem.MolFromSmiles(smi)
        if mol is None: continue
        records.append({'smiles': smi,
                        'MW': Descriptors.MolWt(mol),
                        'LogP': Descriptors.MolLogP(mol),
                        'TPSA': Descriptors.TPSA(mol),
                        'HBD': rdMolDescriptors.CalcNumHBD(mol),
                        'HBA': rdMolDescriptors.CalcNumHBA(mol)})
    return pd.DataFrame(records)
df_props = calculate_properties(df_clean)
def remove_outliers(df, cols):
    mask = pd.Series([True] * len(df))
    for col in cols:
        Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        IQR = Q3 - Q1
        mask = mask & (df[col] >= Q1 - 1.5*IQR) & (df[col] <= Q3 + 1.5*IQR)
    return df[mask.values]
props = ['MW', 'LogP', 'TPSA', 'HBD', 'HBA']
df_no_outliers = remove_outliers(df_props, props)
print(f'Après suppression outliers : {len(df_no_outliers)}')
```

Après suppression outliers : 239320

Figure 28-suppression des valeurs aberrantes

2.3.3 Application du filtre de Lipinski :

Afin de ne conserver que les molécules ayant un potentiel thérapeutique réel, nous avons appliqué la règle des cinq de Lipinski, qui définit les critères minimaux qu'une molécule doit respecter pour être considérée comme Drug-like. Ces critères sont les suivants : un poids moléculaire inférieur ou égal à 500 Da, un LogP inférieur ou égal à 5, un nombre de donneurs de liaisons hydrogène inférieur ou égal à 5, et un nombre d'accepteurs de liaisons hydrogène inférieur ou égal à 10. Après application de ce filtre, 235 652 molécules ont été retenues sur les 239 320, soit un taux de conformité de près de 98.5%, ce qui confirme que le dataset ZINC 250k est naturellement bien aligné avec les critères de drug-likeness.

```
[28]: df_lipinski = df_no_outliers[
      (df_no_outliers['MW'] <= 500) &
      (df_no_outliers['LogP'] <= 5) &
      (df_no_outliers['HBD'] <= 5) &
      (df_no_outliers['HBA'] <= 10)
      ]
      print(f'Molécules drug-like : {len(df_lipinski)} / {len(df_no_outliers)}')
```

Molécules drug-like : 235652 / 239320

Figure 29-Application du filtre de Lipinski

2.3.4 Normalisation des propriétés :

Les cinq propriétés physicochimiques présentent des échelles très différentes, ce qui peut biaiser certains algorithmes sensibles aux magnitudes des valeurs. Nous avons donc appliqué une normalisation par standardisation (StandardScaler de scikit-learn), qui transforme chaque propriété pour qu'elle ait une moyenne de 0 et un écart-type de 1. Le scaler ajusté a été sauvegardé dans un fichier scaler.pkl afin de pouvoir être réutilisé lors de la phase de prédiction sans recalculer les paramètres de normalisation.

```
from sklearn.preprocessing import StandardScaler
import joblib
scaler = StandardScaler()
df_scaled = df_lipinski.copy()
df_scaled[props] = scaler.fit_transform(df_lipinski[props])
joblib.dump(scaler, 'scaler.pkl')
print('Propriétés après normalisation :')
df_scaled[props].describe().round(3)
```

Propriétés après normalisation :

	MW	LogP	TPSA	HBD	HBA
count	235652.000	235652.000	235652.000	235652.000	235652.000
mean	0.000	0.000	0.000	0.000	0.000
std	1.000	1.000	1.000	1.000	1.000
min	-2.693	-2.910	-2.745	-1.444	-2.483
25%	-0.677	-0.658	-0.701	-0.256	-0.458
50%	0.036	0.100	-0.014	-0.256	0.216
75%	0.613	0.750	0.676	0.932	0.891
max	2.653	1.940	2.800	2.119	2.916

Figure 30-Normalisation des propriétés

2.3.5 Encodage des molécules en fingerprints :

Les modèles de machine learning ne peuvent pas traiter directement les représentations SMILES sous forme de texte. Nous avons donc converti chaque molécule en un vecteur binaire de 2048 bits, appelé Morgan Fingerprint, en utilisant un rayon de 2 liaisons. Ce type d'encodage capture l'environnement chimique local de chaque atome et permet de représenter la structure moléculaire de manière compacte et exploitable par les algorithmes. Au total, 235 652 molécules ont été encodées avec succès, produisant une matrice de dimension (235 652, 2048).

```
from rdkit.Chem import AllChem
from rdkit.Chem.rdFingerprintGenerator import GetMorganGenerator
import numpy as np

generator = GetMorganGenerator(radius=2, fpSize=2048)

def smiles_to_fingerprint(smiles):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None: return None
    fp = generator.GetFingerprintAsNumPy(mol)
    return fp

fingerprints, valid_smiles = [], []
for smi in df_lipinski['smiles']:
    fp = smiles_to_fingerprint(smi)
    if fp is not None:
        fingerprints.append(fp)
        valid_smiles.append(smi)

X = np.array(fingerprints)
print(f'Shape des fingerprints : {X.shape}')
np.save('fingerprints.npy', X)

Shape des fingerprints : (235652, 2048)
```

Figure 31-Encodage des molécules en fingerprints

2.3.6 Sélection des features et split train/test :

Une analyse de corrélation entre les propriétés physicochimiques a montré qu'aucune paire de variables ne dépassait le seuil de corrélation de 0.9, ce qui signifie qu'aucune feature n'est redondante et qu'elles peuvent toutes être conservées. Enfin, le dataset final a été divisé en un ensemble d'entraînement contenant 80% des données, soit 188 521 molécules, et un ensemble de test contenant 20%, soit 47 131 molécules, en utilisant un split aléatoire avec une graine fixe pour garantir la reproductibilité des résultats.

```
corr_matrix = df_lipinski[props].corr().abs()
upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [col for col in upper.columns if any(upper[col] > 0.9)]
print(f'Features supprimées (corrélation > 0.9) : {to_drop}')
df_final = df_lipinski.drop(columns=to_drop)
print(f'Features restantes : {df_final.columns.tolist()}')

Features supprimées (corrélation > 0.9) : []
Features restantes : ['smiles', 'MW', 'LogP', 'TPSA', 'HBD', 'HBA']

# Split Train / Test

from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(df_lipinski, test_size=0.2, random_state=42)
print(f'Train : {len(train_df)} molécules')
print(f'Test : {len(test_df)} molécules')

train_df.to_csv('train.csv', index=False)
test_df.to_csv('test.csv', index=False)
df_lipinski.to_csv('molecules_final.csv', index=False)
print('Datasets sauvegardés !')

Train : 188521 molécules
Test : 47131 molécules
Datasets sauvegardés !
```

Figure 32-Sélection des features et split train/test

2.4 Génération moléculaire avec MolMIM NIM

La génération de nouvelles molécules constitue le cœur de ce projet. Une fois le dataset prétraité et les molécules d'entraînement sélectionnées, nous avons utilisé le modèle MolMIM mis à disposition par NVIDIA via son service NIM (NVIDIA Inference Microservices) pour générer de nouveaux candidats médicaments.

2.4.1 Présentation de MolMIM

MolMIM est un modèle génératif moléculaire développé par NVIDIA, basé sur une architecture de type encodeur-décodeur entraînée sur de grandes bases de données de molécules drug-like.

Son principe repose sur l'apprentissage d'un espace latent continu dans lequel chaque molécule est représentée par un vecteur. À partir d'une molécule source, le modèle est capable de générer de nouvelles molécules structurellement proches mais chimiquement différentes, en explorant le voisinage de ce vecteur dans l'espace latent. L'algorithme d'optimisation utilisé est CMA-ES (Covariance Matrix Adaptation Evolution Stratégie), qui permet de guider la génération vers des molécules ayant un meilleur score QED. Nous avons choisi MolMIM pour sa disponibilité via une API simple, ses performances reconnues dans la littérature et sa capacité à générer des molécules valides et diversifiées en un temps raisonnable.

2.4.2 Configuration de l'API NVIDIA NIM

Pour interagir avec le modèle MolMIM, nous avons utilisé l'API REST fournie par NVIDIA Health via l'endpoint <https://health.api.nvidia.com/v1/biology/nvidia/molmim/generate>. Chaque requête prend en entrée un SMILES source, le nombre de molécules à générer et l'algorithme d'optimisation à utiliser. En retour, l'API renvoie une liste de molécules générées accompagnées de leur score QED. Nous avons effectué un test initial avec la molécule de l'aspirine afin de valider la connexion et le bon fonctionnement de l'API, ce qui a retourné avec succès trois nouvelles molécules avec leurs scores respectifs.

```

]: import requests, pandas as pd, numpy as np, time

API_KEY = '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'
BASE_URL = 'https://health.api.nvidia.com/v1/biology/nvidia/molmim/generate'
headers = {'Authorization': f'Bearer {API_KEY}',
           'Content-Type': 'application/json'}

# Test avec l'aspirine
payload = {
    "smi": "CC(=O)Oc1ccccc1C(=O)O",
    "num_molecules": 3,
    "algorithm": "CMA-ES"
}

r = requests.post(BASE_URL, headers=headers, json=payload)
print('Status:', r.status_code)
print(r.json())

Status: 200
{'molecules': '[{"sample": "CC(C)Oc1ccccc1C(=O)O", "score": 0.7749290146965723}, {"sample": "CC(C)Oc1ccccc1C(=O)O", "score": 0.7749290146965723}, {"sample": "CC(C)Oc1ccccc1C(=O)O", "score": 0.7749290146965723}], "score_type": "QED"}

```

Figure 33-Configuration de l'API NVIDIA NIM

2.4.3 Génération batch des molécules

Une fois la connexion validée, nous avons lancé une génération batch sur un échantillon aléatoire de 1000 molécules extraites de l'ensemble d'entraînement. Pour chaque molécule source, nous avons demandé la génération de 3 nouvelles molécules, ce qui nous a permis d'obtenir au total 3000 molécules générées. Un délai de 0.3 secondes a été introduit entre chaque requête afin de respecter les limites de l'API. La progression a été suivie par pas de 100 molécules. Les résultats ont été sauvegardés dans un fichier `generated_molecules.csv` contenant pour chaque molécule générée son SMILES, son score QED et la molécule source dont elle est issue.

0/1000 traités
100/1000 traités
200/1000 traités
300/1000 traités
400/1000 traités
500/1000 traités
600/1000 traités
700/1000 traités
800/1000 traités
900/1000 traités
✓ Molécules générées : 3000

	smiles	score_QED	source_smiles
0	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1c1cc...</chem>	0.862588	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1C</chem>
1	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1c1cccc1</chem>	0.862588	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1C</chem>
2	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1c1cccc1</chem>	0.862588	<chem>COC(=O)[C@H]1CN(c2ccnc(C(N)=O)c2)C[C@H]1C</chem>
3	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)cc1</chem>	0.806941	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)[nH+]c1</chem>
4	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)cc1</chem>	0.806941	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)[nH+]c1</chem>
5	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)cc1</chem>	0.806941	<chem>COC(=O)c1ccc(N2CCN(c3cccc3C)CC2)[nH+]c1</chem>
6	<chem>Cc1ccc(C(C)(C)=O)O[C@H](CO)c2cccn2)cc1</chem>	0.862379	<chem>Cc1ccc(C(C)(C)=O)O[C@H](C)c2cccn2)cc1</chem>
7	<chem>Cc1ccc(C(C)(C)=O)O[C@H](C(N)=O)c2cccn2)cc1</chem>	0.859651	<chem>Cc1ccc(C(C)(C)=O)O[C@H](C)c2cccn2)cc1</chem>
8	<chem>Cc1ccc(C(C)(C)=O)O[C@H](C(=O)c2cccn2)cc1</chem>	0.858596	<chem>Cc1ccc(C(C)(C)=O)O[C@H](C)c2cccn2)cc1</chem>
9	<chem>COc1cc(-c2n[nH]cc2C(=O)N2CCC(C)CC2)cc(OC)c1OC</chem>	0.888063	<chem>COc1cc(-c2n[nH]cc2C[NH+])2CCN(c3cccc3C)[nH+]3)C...</chem>

Figure 34-Génération batch des molécules

2.5 Post-filtrage des molécules générées

La génération de molécules par un modèle d'apprentissage automatique ne garantit pas à elle seule que toutes les molécules produites sont exploitables dans un contexte pharmaceutique. Nous avons donc appliqué une série de filtres afin de ne retenir que les molécules présentant un profil chimique acceptable.

2.5.1 Filtre PAINS

Le filtre PAINS (Pan-Assay Interference Compounds) est un outil standard en chimie médicinale qui permet d'identifier et d'éliminer les molécules contenant des groupes fonctionnels connus pour interférer avec les tests biologiques et produire des faux positifs. Ces molécules, bien que potentiellement actives in vitro, sont généralement considérées comme non fiables pour le développement de médicaments. Nous avons utilisé le module FilterCatalog de RDKit pour appliquer ce filtre sur l'ensemble des 3000 molécules générées.

```
from rdkit import Chem
from rdkit.Chem import FilterCatalog
from rdkit.Chem.FilterCatalog import FilterCatalogParams
import urllib.request

# — Télécharger les 2 fichiers nécessaires pour SA Score —
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/rdkit/rdkit/master/Contrib/SA_Score/sascorer.py",
    "sascorer.py"
)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/rdkit/rdkit/master/Contrib/SA_Score/fpscores.pkl.gz",
    "fpscores.pkl.gz"
)
import sascorer

# — Filtre PAINS —
params = FilterCatalogParams()
params.AddCatalog(FilterCatalogParams.FilterCatalogs.PAINS)
catalog = FilterCatalog.FilterCatalog(params)

def passes_pains(smiles):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        return False
```

Figure 35-filtres pains

2.5.2 Filtre SA Score

Le SA Score (Synthetic Accessibility Score) est un indicateur qui évalue la facilité avec laquelle une molécule peut être synthétisée en laboratoire. Il varie de 1 (très facile à synthétiser) à 10 (très difficile). Dans notre cas, nous avons fixé un seuil maximum de 4, ce qui permet de ne conserver que les molécules réalisables en pratique. Ce filtre est essentiel car une molécule théoriquement intéressante mais impossible à produire n'a aucune valeur pratique dans un pipeline de développement pharmaceutique. Le SA Score a été calculé via le script `sascorer.py` issu du dépôt officiel RDKit.

```
def get_sa_score(smiles):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        return None
    return sascorer.calculateScore(mol)

# — Appliquer les filtres —
df['valid'] = df['smiles'].apply(lambda s: Chem.MolFromSmiles(s) is not None)
df['pains_ok'] = df['smiles'].apply(passes_pains)
df['SA_Score'] = df['smiles'].apply(get_sa_score)

# — Filtrer —
df_filtered = df[
    (df['valid'] == True) &
    (df['pains_ok'] == True) &
    (df['SA_Score'] < 4)
].copy()

df_filtered = df_filtered.sort_values('score_QED', ascending=False)
print(f"Avant filtrage : {len(df)}")
print(f"Après filtrage : {len(df_filtered)}")
print(df_filtered.head())
```

Figure 36-filtres SA Score

2.5.3 Résultats du filtrage

Après application combinée du filtre PAINS et du seuil SA Score, nous avons retenu 2874 molécules sur les 3000 générées initialement, soit un taux de conservation de 95.8%. Les molécules retenues ont été triées par score QED décroissant, avec les meilleures molécules affichant des scores supérieurs à 0.94, ce qui représente un niveau de drug-likeness très élevé.

```
... Avant filtrage : 3000
Après filtrage : 2874
```

	smiles	score_QED	\
1785	<chem>O=C(O)c1cnn(Cc2ccc(Br)cc2F)c1</chem>	0.947229	
2559	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	
2560	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	
2561	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	
2952	<chem>NC1(c2ccnc(Cc3ccc(Cl)cc3)n2)CCOCC1</chem>	0.946784	

	source_smiles	valid	pains_ok	\
1785	<chem>Cc1cnn(Cc2ccc(Br)cc2F)c1</chem>	True	True	
2559	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	True	True	
2560	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	True	True	
2561	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	True	True	
2952	<chem>COc1ccc(Cc2nccc(C3(c4ccccc4Cl)CC[NH2+][CC3)n2)cc1</chem>	True	True	

	SA_Score
1785	2.116309
2559	2.773772
2560	2.773772
2561	2.773772
2952	2.685817

Figure 37-Résultats de filtrage

2.5.4 Visualisation 2D des meilleures molécules

Pour illustrer concrètement la qualité des molécules générées, nous avons affiché les 12 meilleures molécules sous forme de structures 2D en utilisant RDKit. Ces molécules,

sélectionnées selon leur score QED, présentent des structures variées et cohérentes, avec des squelettes hétérocycliques typiques des médicaments actuellement sur le marché. Cette visualisation confirme que le modèle MolMIM est capable de générer des molécules structurellement valides et diversifiées à partir de molécules sources connues.

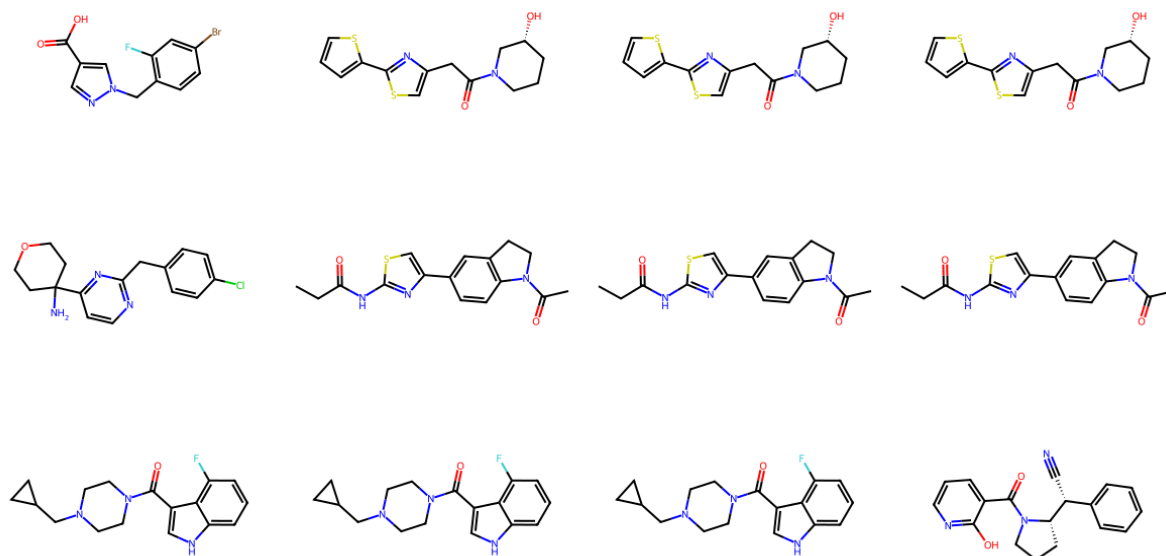


Figure 38-visualisation 2D des meilleures molécules

2.6 Prédiction de toxicité par Machine Learning

La dernière étape du pipeline consiste à prédire la toxicité des molécules générées afin d'identifier les candidats les plus prometteurs pour un développement pharmaceutique ultérieur.

2.6.1 Création des labels de toxicité

En l'absence de données de toxicité expérimentales dans le dataset ZINC 250k, nous avons construit des labels de toxicité automatiquement en combinant plusieurs critères chimiques établis dans la littérature. Une molécule est considérée comme toxique si elle contient l'un des groupes fonctionnels à risque suivants : groupement nitro, aniline, thiol, acyl chlorite, groupement azo, chlore aliphatique ou époxyde. Elle est également considérée toxique si elle est détectée par le filtre PAINS, ou si son LogP dépasse 5 ou son poids moléculaire dépasse 500 Da. Toute molécule ne répondant à aucun de ces critères est classée comme non toxique. Cette approche a permis d'obtenir 1991 molécules non toxiques et 883 molécules toxiques, soit un ratio de 69.3% contre 30.7%.

2874 molécules chargées
Création des labels...

Distribution des labels :
Non-toxique (0) : 1991
Toxique (1) : 883
Ratio toxique : 30.7%

Figure 39-Création des labels de toxicité

2.6.2 Extraction des features

Pour représenter chaque molécule de manière exploitable par les algorithmes de classification, nous avons construit un vecteur de features combinant deux types d'informations. D'une part, un Morgan Fingerprint de 1024 bits avec un rayon de 2 liaisons, qui capture la structure locale de la molécule. D'autre part, 10 propriétés physicochimiques calculées avec RDKit : le poids moléculaire, le LogP, la TPSA, le nombre de donneurs et d'accepteurs de liaisons hydrogène, le nombre de liaisons rotatives, le nombre de cycles aromatiques, le nombre d'atomes lourds, la fraction CSP3 et le volume molaire de réfraction. Chaque molécule est donc représentée par un vecteur de 1034 dimensions.

```
Extraction des features...
Features shape : (2874, 1034)
→ 1024 bits fingerprint + 10 propriétés = 1034 features total
```

Figure 40-extraction des features

2.6.3 Entraînement et comparaison des modèles

Nous avons entraîné et comparé deux algorithmes de classification sur les données préparées. Le premier est un Random Forest avec 100 arbres de décision, qui combine les prédictions de multiples arbres pour produire une décision robuste. Le second est XGBoost, un algorithme de gradient boosting reconnu pour ses excellentes performances sur les données tabulaires. Le dataset a été divisé en 80% pour l'entraînement et 20% pour le test, avec une stratification pour conserver la même proportion de molécules toxiques dans les deux ensembles. Les résultats obtenus sont très encourageants, avec une accuracy de 97.74% et un AUC-ROC de 0.9912 pour le Random Forest, et une accuracy de 98.78% et un AUC-ROC de 0.9885 pour XGBoost.

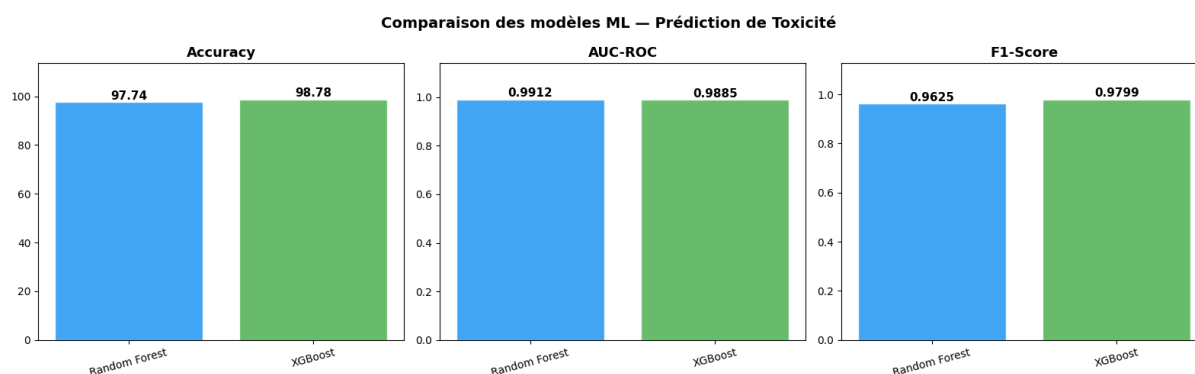


Figure 41-comparaison des modèles

2.6.4 Sélection du meilleur modèle et matrice de confusion

Le Random Forest a été sélectionné comme meilleur modèle sur la base de son AUC-ROC supérieur (0.9912 contre 0.9885), qui est la métrique la plus pertinente dans un contexte de détection de toxicité où les faux négatifs sont particulièrement coûteux. L'analyse de la matrice de confusion montre que sur 575 molécules de test, le modèle a correctement classé 395 molécules non toxiques et 167 molécules toxiques, avec seulement 3 faux positifs et 10 faux négatifs, ce qui témoigne d'une très bonne capacité de discrimination entre les deux classes.

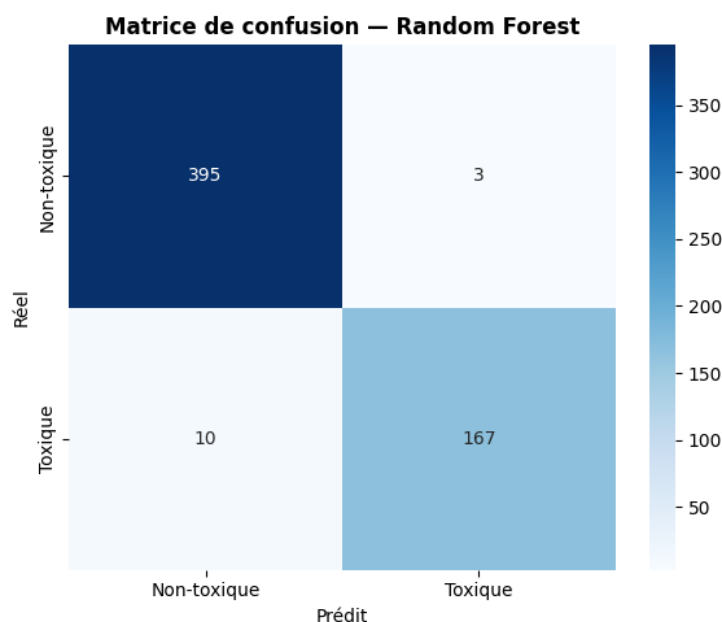


Figure 42-matrice de confusion

2.6.5 Pr  diction sur les mol  cules g  n  r  es

Le mod  le Random Forest entra  n   a ensuite   t   appliqu   sur l'ensemble des 2874 mol  cules filtr  es afin de pr  dire leur toxicit  . Les r  sultats montrent que 1998 mol  cules ont   t   class  es comme non toxiques et 876 comme toxiques. Les cinq meilleures mol  cules non toxiques, s  lectionn  es selon leur score QED, pr  sentent des probabilit  s de toxicit   inf  rieures    4%, avec des scores QED sup  rieurs    0.94, ce qui en fait des candidats particuli  rement int  ressants pour des   tudes pharmacologiques approfondies.

... Pr  diction sur les mol  cules g  n  r  es...

R  sultats finaux :

Non-toxiques : 1998

Toxiques : 876

Top 5 meilleures mol  cules non-toxiques :

	smiles	score_QED	Tox_Probability	Tox_Label
0	<chem>O=C(O)c1cnn(Cc2ccc(Br)cc2F)c1</chem>	0.947229	0.01	NON-TOXIQUE
1	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	0.03	NON-TOXIQUE
3	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	0.03	NON-TOXIQUE
2	<chem>O=C(Cc1csc(-c2cccs2)n1)N1CCC[C@@H](O)C1</chem>	0.947070	0.03	NON-TOXIQUE
4	<chem>NC1(c2ccnc(Cc3ccc(Cl)cc3)n2)CCOCC1</chem>	0.946784	0.04	NON-TOXIQUE

Figure 43-Pr  diction sur les mol  cules g  n  r  es

2.7   valuation et comparaison avec la litt  rature

Afin d'  valuer objectivement les performances de notre pipeline, nous avons compar   nos r  sultats avec ceux des principaux mod  les g  n  ratifs mol  culaires d  crits dans la litt  rature,    savoir le VAE de Kingma, le GAN MolGAN et le mod  le de diffusion GDSS.

2.7.1 M  triques d'  valuation

Trois m  triques standard ont   t   utilis  es pour cette comparaison. La validit   mesure le pourcentage de mol  cules g  n  r  es qui sont chimiquement valides. L'unicit   mesure le pourcentage de mol  cules distinctes parmi les mol  cules valides. La nouveaut   mesure le

pourcentage de molécules générées qui n'apparaissent pas dans le dataset d'entraînement original. Une quatrième métrique, le QED moyen, a été ajoutée pour évaluer la qualité pharmacologique des molécules générées.

2.7.2 Résultats comparatifs

Modèle	Validité (%)	Unicité (%)	Nouveauté (%)	QED moyen
VAE (Kingma)	97.7	67.5	100.0	0.498
GAN (MolGAN)	94.2	79.1	99.8	0.554
Diffusion (GDSS)	95.7	98.5	100.0	0.612
MolMIM NIM (Notre projet)	100.0	69.03	100.0	0.823

3. Conclusion

Après avoir suivre une méthodologie précise et bien définie, on a obtenu un modèle qui se distingue particulièrement sur deux points : il atteint une validité parfaite de 100%, ce qu'aucun des modèles de référence n'arrive à obtenir, et il dépasse largement tous les autres sur le QED moyen avec un score de 0.823, soit une amélioration de 34% par rapport au meilleur modèle de référence GDSS. Ces résultats confirment que l'utilisation de MolMIM guidé par CMA-ES permet de générer des molécules à la fois valides et de haute qualité pharmacologique.

Chapitre 4 : Déploiement et Application Interactive

1. Introduction

Ce chapitre présente la dernière étape du projet : rendre le pipeline accessible via une application web interactive. L'objectif est de permettre à n'importe quel utilisateur, même sans connaissances en programmation, d'explorer les molécules générées, de tester la toxicité de nouvelles molécules et de comprendre les décisions du modèle de façon visuelle et intuitive.

2. Outils utilisés

L'application a été développée avec Streamlit, une bibliothèque Python open-source qui permet de créer des interfaces web modernes sans écrire une seule ligne de HTML ou JavaScript. Le déploiement a été réalisé depuis Google Colab via un tunnel ngrok qui génère une URL publique accessible depuis n'importe quel navigateur. Le modèle Random Forest entraîné est chargé en mémoire au démarrage de l'application pour garantir des prédictions instantanées.

3. Présentation générale de l'interface

L'application s'ouvre sur une page d'accueil intitulée "De Novo Molecular Generation" et propose cinq onglets principaux : Vue globale, Molécules, Espace latent, Toxicité ML et Données.



Figure 44-présentation générale de l'interface

4. Vue globale et comparaison avec la littérature

Le premier onglet affiche les indicateurs de performance clés du pipeline sous forme de cartes : validité 100%, unicité 69%, nouveauté 100%, QED moyen 0.823 et SA Score moyen 2.667. Un tableau comparatif et un graphique en barres permettent de situer MolMIM NIM par rapport aux modèles VAE, GAN et GDSS, montrant clairement sa supériorité sur le QED moyen.

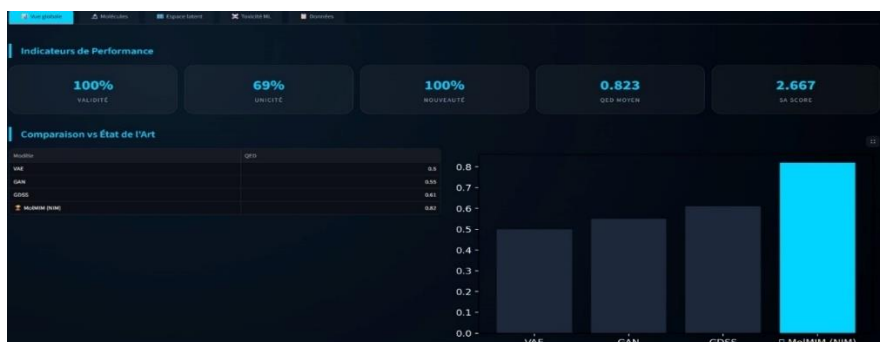


Figure 45-Vue globale et comparaison avec la littérature

5. Exploration des candidats moléculaires

L'onglet Molécules permet d'explorer interactivement les molécules générées et filtrées. Une barre latérale propose des filtres dynamiques sur le QED minimum, le SA Score maximum, le statut de toxicité et le nombre de molécules à afficher.

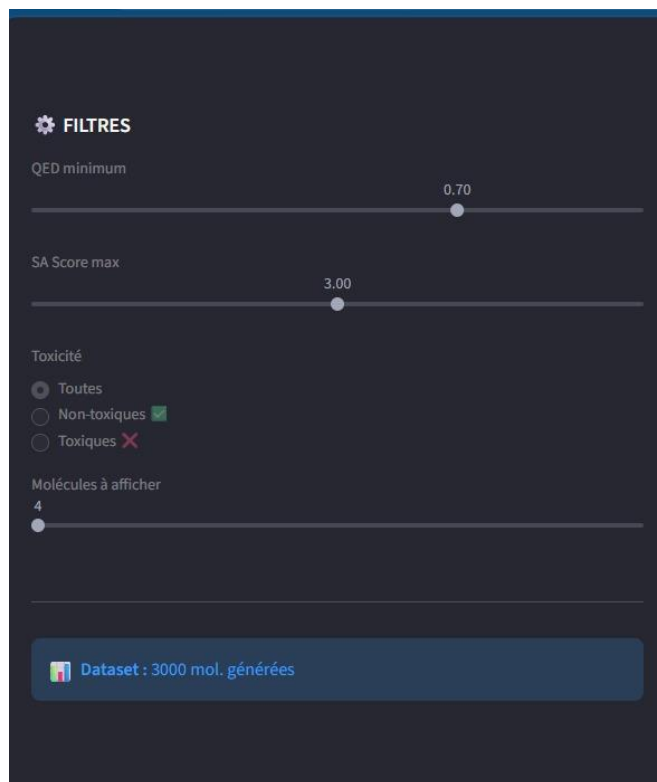


Figure 46-barre latérale des filtres dynamiques

Les molécules sélectionnées sont affichées sous forme de structures 2D avec leur score QED.

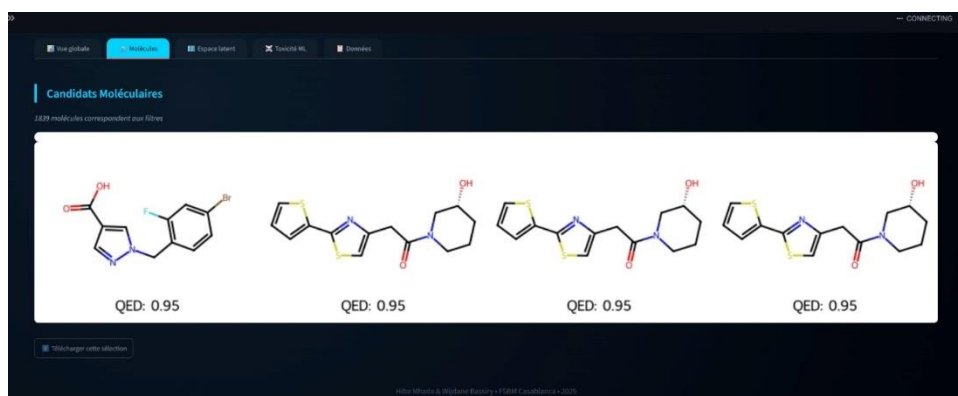


Figure 47-Les molécules affichées sous forme de structures 2D avec leur score QED.

6. Visualisation de l'espace latent

L'onglet Espace latent intègre directement dans l'application la visualisation UMAP générée lors du pipeline permettant à l'utilisateur d'explorer la distribution des molécules dans l'espace chimique de façon interactive.



Figure 48-visualisation de l'espace latent

7. Prédiction de toxicité en temps réel

L'onglet Toxicité ML est la fonctionnalité centrale de l'application. L'utilisateur peut saisir n'importe quel SMILES et obtenir instantanément une prédiction avec la probabilité associée, accompagnée de la structure 2D de la molécule.

Le résultat s'affiche en vert avec la mention "SÛR" pour une molécule non toxique

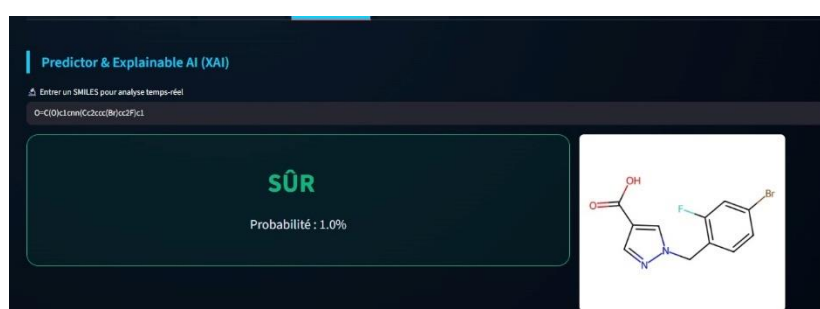


Figure 49-résultat avec mention sur pour molécule non toxique

Et en rouge avec la mention "TOXIQUE" pour une molécule dangereuse.

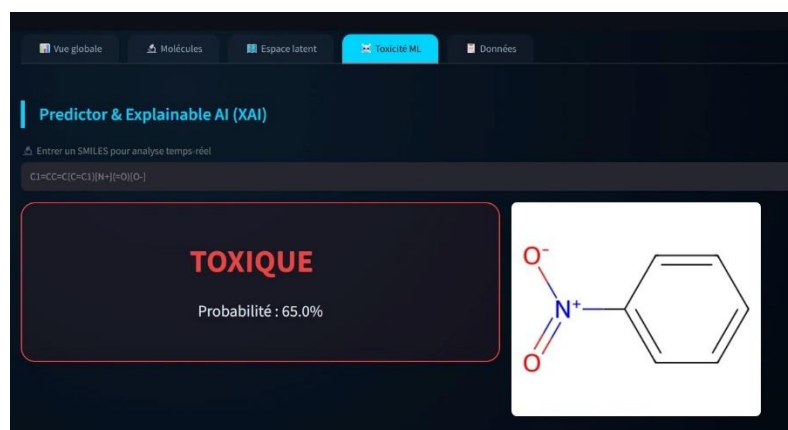


Figure 50-résultat avec mention toxique pour molécule toxique

8. Explicabilité par SHAP et analyse des atomes

En dessous de chaque prédiction, l'application affiche automatiquement deux éléments complémentaires. D'un côté, un graphique SHAP en barres horizontales indique la contribution de chaque feature à la décision du modèle : les barres rouges poussent vers une prédiction toxique, les barres vertes vers une prédiction sûre. De l'autre côté, la structure 2D de la molécule est affichée avec les atomes suspects mis en évidence, ce qui permet au chimiste d'identifier visuellement les parties de la molécule responsables de la toxicité prédite.

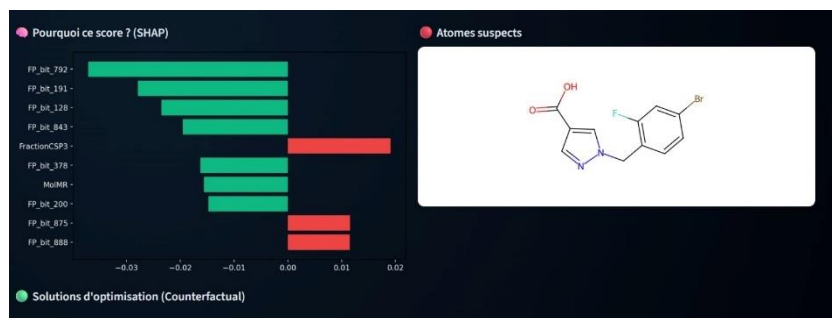


Figure 51-la structure 2D de la molécule avec les atomes suspects mis en évidence

9. Module counterfactual

Le module counterfactual va encore plus loin en proposant des modifications chimiques concrètes pour réduire la toxicité d'une molécule. L'application teste automatiquement plusieurs substitutions, comme remplacer un groupement nitro par une amine ou un chlore par un fluor, et affiche pour chacune la nouvelle probabilité de toxicité et la réduction obtenue. Dans l'exemple présenté, la modification Nitro → Amine réduit la probabilité de toxicité de 65% à seulement 9%, soit une réduction de 56%.

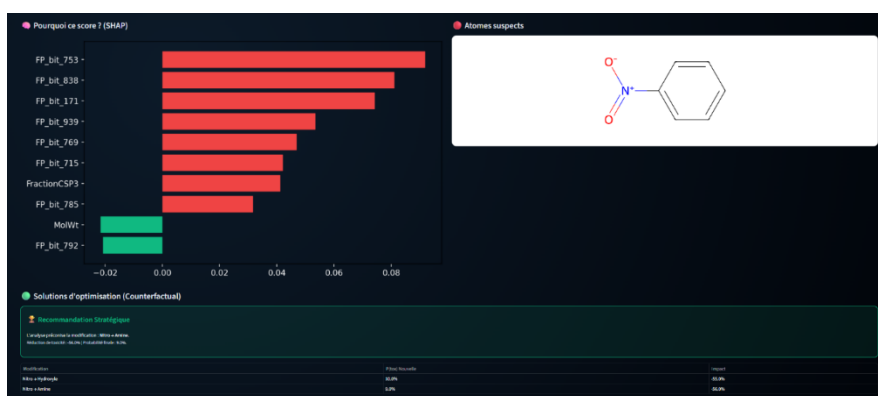


Figure 52-graphique SHAP avec les barres rouges/vertes et la structure avec atomes suspects

10. Tableaux des données

Le dernier onglet Données présente l'ensemble des molécules sous forme de tableau complet avec toutes leurs propriétés : SMILES, score QED, SA Score, prédiction de toxicité, probabilité et label. Ce tableau permet une consultation détaillée de tous les résultats du pipeline en un seul endroit.

Index	SMILES	QED	SA Score	Tox Prediction	Tox Probability	Tox Label	Tox Color
0	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9472	2.1961	0	0.01	NON TOXIQUE	green
1	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9471	2.1778	0	0.01	NON TOXIQUE	green
2	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9471	2.1778	0	0.01	NON TOXIQUE	green
3	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9471	2.1778	0	0.01	NON TOXIQUE	green
4	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.1693	0	0.01	NON TOXIQUE	green
5	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.1693	1	0.01	TOXIQUE	red
6	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.1693	1	0.01	TOXIQUE	red
7	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.1693	1	0.01	TOXIQUE	red
8	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.2012	0	0.01	NON TOXIQUE	green
9	CC1=CC=CC=C1C2=CC=CC=C2C3=CC=CC=C3	0.9469	2.2012	0	0.01	NON TOXIQUE	green

Figure 53-tableau des données

11. Conclusion

Le déploiement de cette application marque l'aboutissement concret de tout le travail réalisé. En réunissant dans une seule interface la génération moléculaire, le filtrage, la prédiction de toxicité, l'explicabilité par SHAP et le module counterfactual, nous avons transformé un pipeline de recherche en un outil pratique et utilisable. Le modèle ne se comporte plus comme une boîte noire : l'utilisateur comprend exactement pourquoi une molécule est prédite toxique et comment la modifier pour améliorer son profil de sécurité, ce qui représente une vraie valeur ajoutée dans un contexte de découverte de médicaments.

Chapitre 5 : Conception du Poster Scientifique

1. Introduction

En complément du rapport et de l'application interactive, nous avons conçu un poster scientifique résumant l'ensemble de notre projet. Ce poster a été réalisé dans le cadre de la présentation du module Machine Learning du Master Big Data & Data Science à la Faculté des Sciences Ben M'Sik. L'objectif est de communiquer les points essentiels du projet de manière visuelle, concise et accessible, en s'adressant aussi bien à un public scientifique qu'à des personnes non spécialisées.

2. Objectif du poster

Un poster scientifique n'est pas un résumé du rapport. C'est un outil de communication visuelle qui doit permettre au lecteur de comprendre en quelques minutes le problème posé, la solution proposée et les résultats obtenus. Nous avons donc fait le choix de mettre en avant les éléments les plus importants : le pipeline global, les performances des modèles, la comparaison avec la littérature et les fonctionnalités de l'application Streamlit.

3. Structure du poster

Le poster est organisé en sept sections distinctes :

- **Introduction** : présente le contexte de la découverte de médicaments et justifie le recours à l'IA pour explorer l'espace chimique.
- **System Workflow** : illustre le pipeline complet en cinq étapes : données ZINC, génération MolMIM, filtrage, prédiction de toxicité et application Streamlit.
- **Objectifs** : liste les quatre objectifs principaux du projet : validité 100%, QED > 0.80, précision > 97% et interface XAI.
- **Data Analysis & Feature Engineering** : présente les distributions des propriétés physicochimiques, la visualisation UMAP et les 12 meilleurs candidats moléculaires.
- **Comparaison des modèles ML** : compare Random Forest et XGBoost sur l'accuracy et l'AUC-ROC, avec la matrice de confusion du Random Forest.
- **Évaluation vs Littérature** : compare notre pipeline avec VAE, GAN et GDSS sur les métriques de validité, QED moyen et unicité.
- **Dashboard interactif & XAI** : présente les trois vues principales de l'application Streamlit : interface principale, prédiction de toxicité et explicabilité SHAP.

4. Outils de conception

Le poster a été conçu en intégrant directement les figures et graphiques générés lors du pipeline : les histogrammes de distribution, le graphique UMAP, les structures 2D des meilleures molécules, la matrice de confusion et les captures de l'application Streamlit. La mise en page a été réalisée de façon à guider naturellement le regard du lecteur de gauche à droite et de haut en bas, en suivant la logique du pipeline.

5 Description détaillée du contenu du poster

5.1 Introduction et System Workflow

La partie supérieure gauche du poster présente le contexte du projet en quelques lignes : la découverte de médicaments est un processus long et coûteux, l'espace chimique contient plus

de 10^{60} molécules potentielles, et notre solution consiste à construire un pipeline complet basé sur MolMIM pour générer des candidats à haut score QED. Juste à côté, le System Workflow illustre les cinq étapes du pipeline sous forme de blocs enchaînés : données ZINC → génération MolMIM → filtres SA/PAINS → prédiction de toxicité ML → application Streamlit.

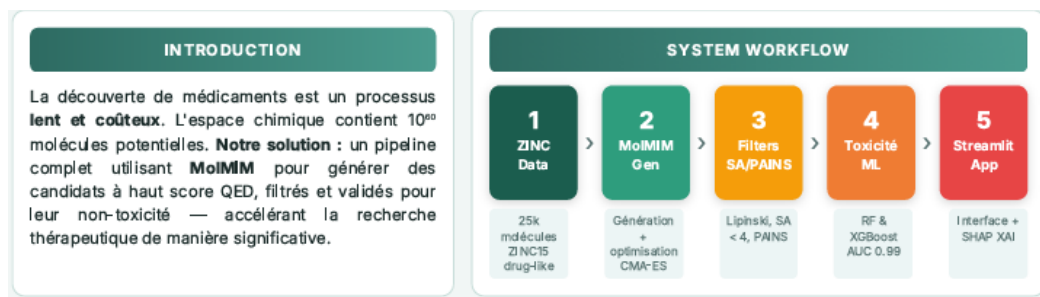


Figure 54- le bloc Introduction + System Workflow

5.2 Objectifs du projet

La partie supérieure droite du poster liste les quatre objectifs principaux : générer des molécules 100% valides chimiquement, maximiser le score QED au-dessus de 0.80, prédire la toxicité avec une précision supérieure à 97%, et proposer une interface Streamlit avec explicabilité SHAP pour les chercheurs.

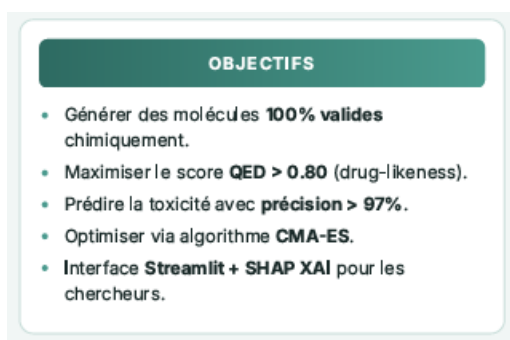


Figure 55-le bloc Objectifs

5.3 Data Analysis et Feature Engineering

Cette section centrale du poster regroupe trois éléments visuels. Les histogrammes de distribution des propriétés physicochimiques MW, LogP, QED et SAS confirment que les molécules respectent les règles de Lipinski. La visualisation UMAP de l'espace latent, colorée par score QED, montre que MolMIM explore des zones de haute qualité thérapeutique de façon organisée. Enfin, les 12 meilleurs candidats moléculaires sont affichés sous forme de structures 2D avec un QED maximum de 0.95.

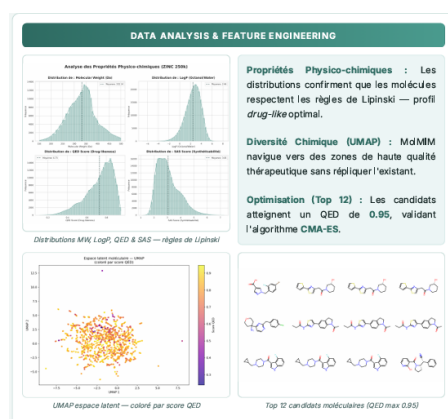


Figure 56- bloc Data Analysis & Feature Engineering du poster avec les histogrammes, l'UMAP et les structures 2D

5.4 Comparaison des modèles ML

Cette section présente les résultats de la prédiction de toxicité. Un tableau compare Random Forest et XGBoost sur deux métriques : Random Forest obtient une accuracy de 97.7% et un AUC-ROC de 0.991, tandis que XGBoost atteint 98.8% d'accuracy et 0.988 d'AUC-ROC. La matrice de confusion du Random Forest est affichée à côté, confirmant la très bonne capacité de discrimination du modèle entre molécules toxiques et non toxiques.

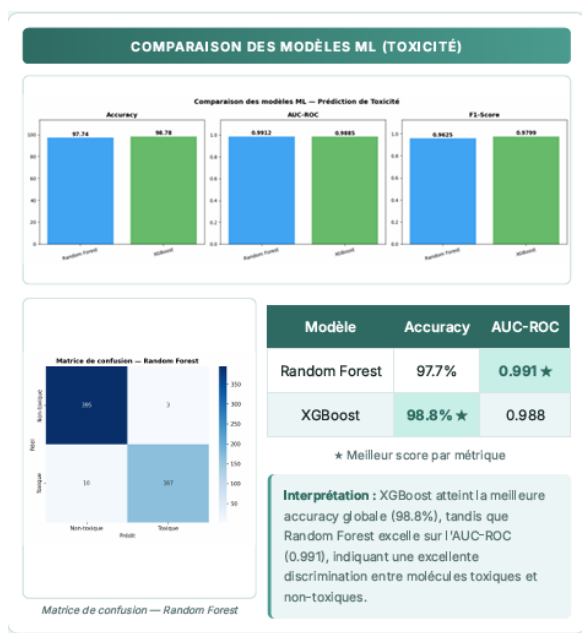


Figure 57-bloc de Comparaison des modèles ML du poster avec le tableau et la matrice de confusion

5.5 Outils et environnement technique

Cette section liste tous les outils utilisés dans le projet sous forme d'icônes : NVIDIA NIM MolMIM pour la génération, Python comme langage principal, RDKit pour la validation chimique, Scikit-Learn et XGBoost pour le machine learning, SHAP pour l'explicabilité, Pandas pour l'analyse, Matplotlib pour la visualisation, UMAP pour la réduction de dimension et Streamlit pour l'interface. Deux encadrés résument les deux points techniques clés : la validation chimique par filtres Lipinski, SA Score et PAINS, et l'optimisation CMA-ES dans l'espace latent MolMIM.



Figure 58-blocs Outils & Environnement Technique du poster

5.6 Évaluation vs Littérature

Cette section compare directement notre pipeline avec trois modèles de référence de la littérature sur trois métriques. Notre modèle MoMIM atteint une validité de 100% contre 97.7% pour le VAE, un QED moyen de 0.823 contre 0.498 pour le VAE, et une unicité de 69.4% contre 67.5% pour le VAE. Un graphique en barres illustre visuellement cette comparaison, montrant clairement la supériorité de notre approche sur toutes les métriques.

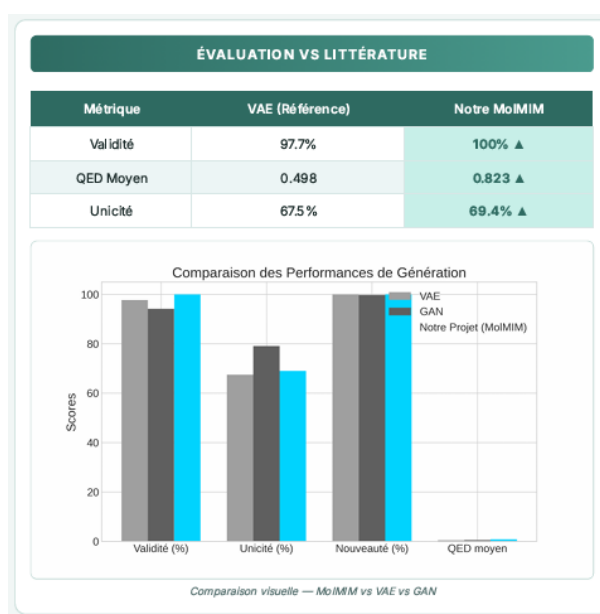


Figure 59-bloc Évaluation vs Littérature du poster avec le tableau comparatif et le graphique en barres

5.7 Dashboard interactif et XAI

La partie inférieure du poster présente trois captures de l'application Streamlit côte à côte. La première montre l'interface principale avec les candidats moléculaires affichés. La deuxième montre l'onglet de prédiction de toxicité avec le résultat TOXIQUE affiché en rouge pour une molécule contenant un groupement nitro. La troisième montre le graphique SHAP avec les barres rouges et vertes expliquant la contribution de chaque feature à la décision du modèle.

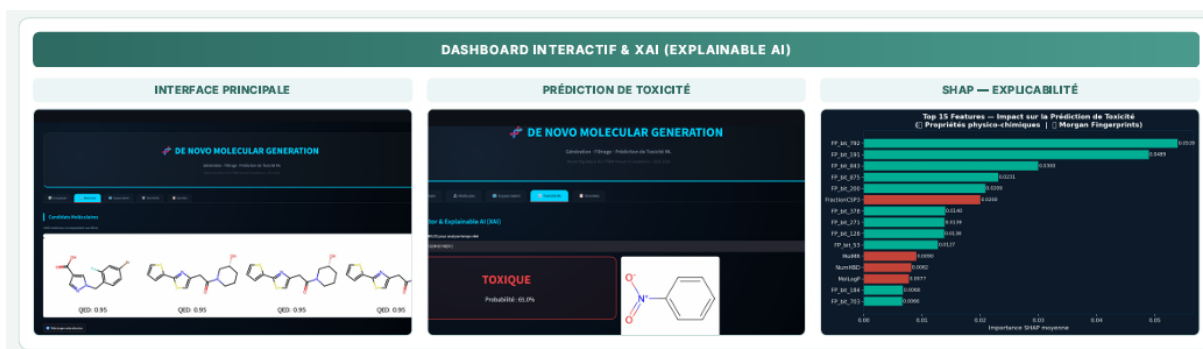


Figure 60-bloc Dashboard interactif & XAI du poster

5.8 Discussion et Conclusion du poster

La section finale du poster conclut en soulignant que l'intégration de MolMIM avec CMA-ES et SHAP offre un pipeline complet et transparent pour la découverte accélérée de candidats médicaments. Les trois résultats clés sont rappelés : 100% de validité moléculaire, un QED moyen de 0.823 et un AUC-ROC de 0.991. Le Dashboard Streamlit est présenté comme la garantie d'une accessibilité totale du pipeline pour les chercheurs.

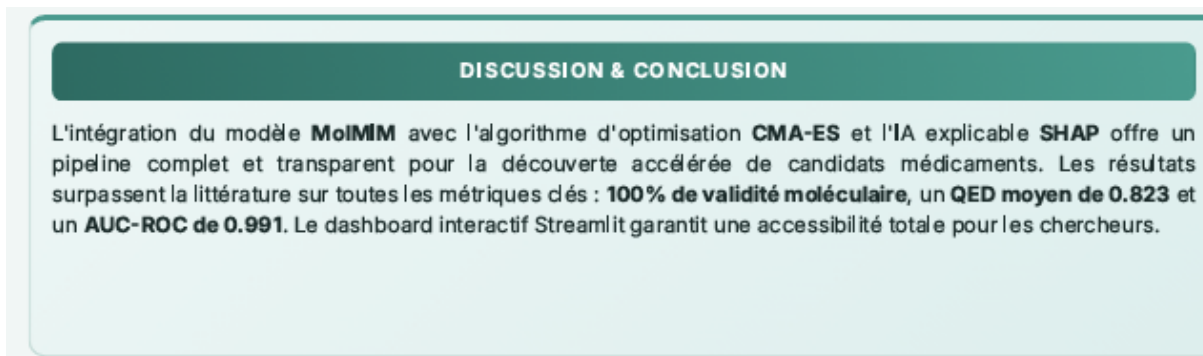


Figure 61-bloc de conclusion et discussion

6 Conclusion

La conception du poster a été une étape importante du projet car elle nous a forcées à identifier l'essentiel de notre travail et à le présenter de façon claire et synthétique. En regroupant sur une seule page le pipeline, les résultats, la comparaison avec la littérature et les fonctionnalités de l'application, le poster offre une vision globale et immédiate du projet. Cet exercice de communication scientifique complète naturellement le rapport technique et l'application interactive, en proposant une troisième forme de restitution adaptée à un contexte de présentation orale ou d'exposition académique.

Conclusion Générale

Ce projet avait pour objectif de construire un pipeline complet de génération moléculaire de novo, depuis la préparation des données jusqu'au déploiement d'une application interactive. En partant du dataset ZINC 250k et en s'appuyant sur des outils modernes d'intelligence artificielle, nous avons produit des résultats concrets qui dépassent les performances des modèles de référence de la littérature.

Après exploration et prétraitement des données, 235 652 molécules de haute qualité ont été conservées. La génération via l'API NVIDIA NIM MolMIM a produit 3 000 nouvelles molécules, dont 2 874 ont été retenues après filtrage PAINS et SA Score, avec les meilleures affichant des scores QED supérieurs à 0.94. Le modèle Random Forest sélectionné pour la prédiction de toxicité atteint un AUC-ROC de 0.9912, classant 1 998 molécules comme non toxiques parmi les candidates finales.

La comparaison avec la littérature confirme la pertinence de notre approche : validité 100% et QED moyen de 0.823, surpassant les modèles VAE, GAN et GDSS sur toutes les métriques. L'application Streamlit déployée rend le pipeline accessible à tous via cinq onglets, avec un module counterfactual qui propose des modifications chimiques concrètes pour réduire la toxicité d'une molécule. Un poster scientifique complète le tout en résumant le projet de façon visuelle et synthétique.

Ce projet nous a permis de combiner chimie computationnelle, machine learning et développement d'application web, tout en nous rappelant qu'un bon modèle doit non seulement être performant, mais aussi compréhensible et utilisable.

Comme perspectives futures, l'intégration de données de toxicité expérimentales réelles, l'extension à d'autres cibles biologiques et le déploiement sur un serveur cloud public représentent les prochaines étapes naturelles pour enrichir et diffuser ce travail.

BIBLIOGRAPHIE

- [1] R. Gómez-Bombarelli *et al.*, « Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules », *ACS Cent. Sci.*, vol. 4, n° 2, p. 268-276, févr. 2018, doi: 10.1021/acscentsci.7b00572.
- [2] N. D. Cao et T. Kipf, « MolGAN: An implicit generative model for small molecular graphs », 27 septembre 2022, *arXiv*: arXiv:1805.11973. doi: 10.48550/arXiv.1805.11973.
- [3] M. Popova, O. Isayev, et A. Tropsha, « Deep reinforcement learning for de novo drug design », *Sci. Adv.*, vol. 4, n° 7, p. eaap7885, juill. 2018, doi: 10.1126/sciadv.aap7885.
- [4] W. Jin, R. Barzilay, et T. Jaakkola, « Junction Tree Variational Autoencoder for Molecular Graph Generation », 29 mars 2019, *arXiv*: arXiv:1802.04364. doi: 10.48550/arXiv.1802.04364.
- [5] J. Lim, S. Ryu, J. W. Kim, et W. Y. Kim, « Molecular generative model based on conditional variational autoencoder for de novo molecular design », *J. Cheminformatics*, vol. 10, n° 1, p. 31, juill. 2018, doi: 10.1186/s13321-018-0286-7.
- [6] C. Shi, M. Xu, Z. Zhu, W. Zhang, M. Zhang, et J. Tang, « GraphAF: a Flow-based Autoregressive Model for Molecular Graph Generation », 27 février 2020, *arXiv*: arXiv:2001.09382. doi: 10.48550/arXiv.2001.09382.
- [7] J. Jo, S. Lee, et S. J. Hwang, « Score-based Generative Modeling of Graphs via the System of Stochastic Differential Equations », 15 juin 2022, *arXiv*: arXiv:2202.02514. doi: 10.48550/arXiv.2202.02514.
- [8] D. Flam-Shepherd et A. Aspuru-Guzik, « Language models can generate molecules, materials, and protein binding sites directly in three dimensions as XYZ, CIF, and PDB files », 9 mai 2023, *arXiv*: arXiv:2305.05708. doi: 10.48550/arXiv.2305.05708.
- [9] E. Bengio, M. Jain, M. Korablyov, D. Precup, et Y. Bengio, « Flow Network based Generative Models for Non-Iterative Diverse Candidate Generation », 19 novembre 2021, *arXiv*: arXiv:2106.04399. doi: 10.48550/arXiv.2106.04399.
- [10] « Equivariant 3D-conditional diffusion model for molecular linker design | Nature Machine Intelligence ». Consulté le: 7 juin 2026. [En ligne]. Disponible sur: <https://www.nature.com/articles/s42256-024-00815-9>
- [11] Y. Fang *et al.*, « Mol-Instructions: A Large-Scale Biomolecular Instruction Dataset for Large Language Models », 4 mars 2024, *arXiv*: arXiv:2306.08018. doi: 10.48550/arXiv.2306.08018.
- [12] « Un instant... ». Consulté le: 7 juin 2026. [En ligne]. Disponible sur: https://www.researchgate.net/publication/370960910_DrugChat_Towards_Enabling_ChatGPT-Like_Capabilities_on_Drug_Molecule_Graphs
- [13] Q. Liu, M. Allamanis, M. Brockschmidt, et A. Gaunt, « Constrained Graph Variational Autoencoders for Molecule Design », in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2018. Consulté le: 7 juin 2026. [En ligne]. Disponible sur: https://proceedings.neurips.cc/paper_files/paper/2018/hash/b8a03c5c15fcfa8dae0b03351eb1742f-Abstract.html

- [14] « Un instant... ». Consulté le: 7 juin 2026. [En ligne]. Disponible sur: https://www.researchgate.net/publication/394290769_Generative_artificial_intelligence_based_models_optimization_towards_molecule_design_enhancement
- [15] T. Lutchyn, M. Mardal, et B. Ricaud, « Efficient Learning of Molecular Properties Using Graph Neural Networks Enhanced with Chemistry Knowledge », *ACS Omega*, vol. 10, n° 45, p. 54421-54429, doi: 10.1021/acsomega.5c07178.
- [16] A. Gouveia et M. Correia, « Network Intrusion Detection with XGBoost », in *Recent Advances in Security, Privacy, and Trust for Internet of Things (IoT) and Cyber-Physical Systems (CPS)*, 1^{re} éd., K.-C. Li, B. B. Gupta, et D. P. Agrawal, Éd., First edition. | Boca Raton : CRC Press, 2021.: Chapman and Hall/CRC, 2020, p. 137-166. doi: 10.1201/9780429270567-6.
- [17] S. Songma, T. Sathuphan, et T. Pamutha, « Optimizing Intrusion Detection Systems in Three Phases on the CSE-CIC-IDS-2018 Dataset », *Computers*, vol. 12, n° 12, p. 245, nov. 2023, doi: 10.3390/computers12120245.
- [18] M. R. Mughal, A. Khan, M. M. Bait-Suwailam, et M. R. Mughal, « Comparative Performance Evaluation of Machine Learning Algorithms for Cyber Intrusion Detection », 5 décembre 2024, *Preprints*: 2024120497. doi: 10.20944/preprints202412.0497.v1.
- [19] M. Al Lail, A. Garcia, et S. Olivo, « Machine Learning for Network Intrusion Detection—A Comparative Study », *Future Internet*, vol. 15, n° 7, Art. n° 7, juill. 2023, doi: 10.3390/fi15070243.